18-740: Modern Computer Architecture & Design

Lecture 6:
"Dynamic Instruction Scheduling Techniques"

John Paul Shen
September 10, 2026

- Required Reading Assignments:
  - **Chapter 5 and Chapter 7 of Shen and Lipasti (SnL).**
- Highly Recommended References:
  - http://en.Wikipedia.org/wiki/Pentium_Pro

Electrical & Computer ENGINEERING

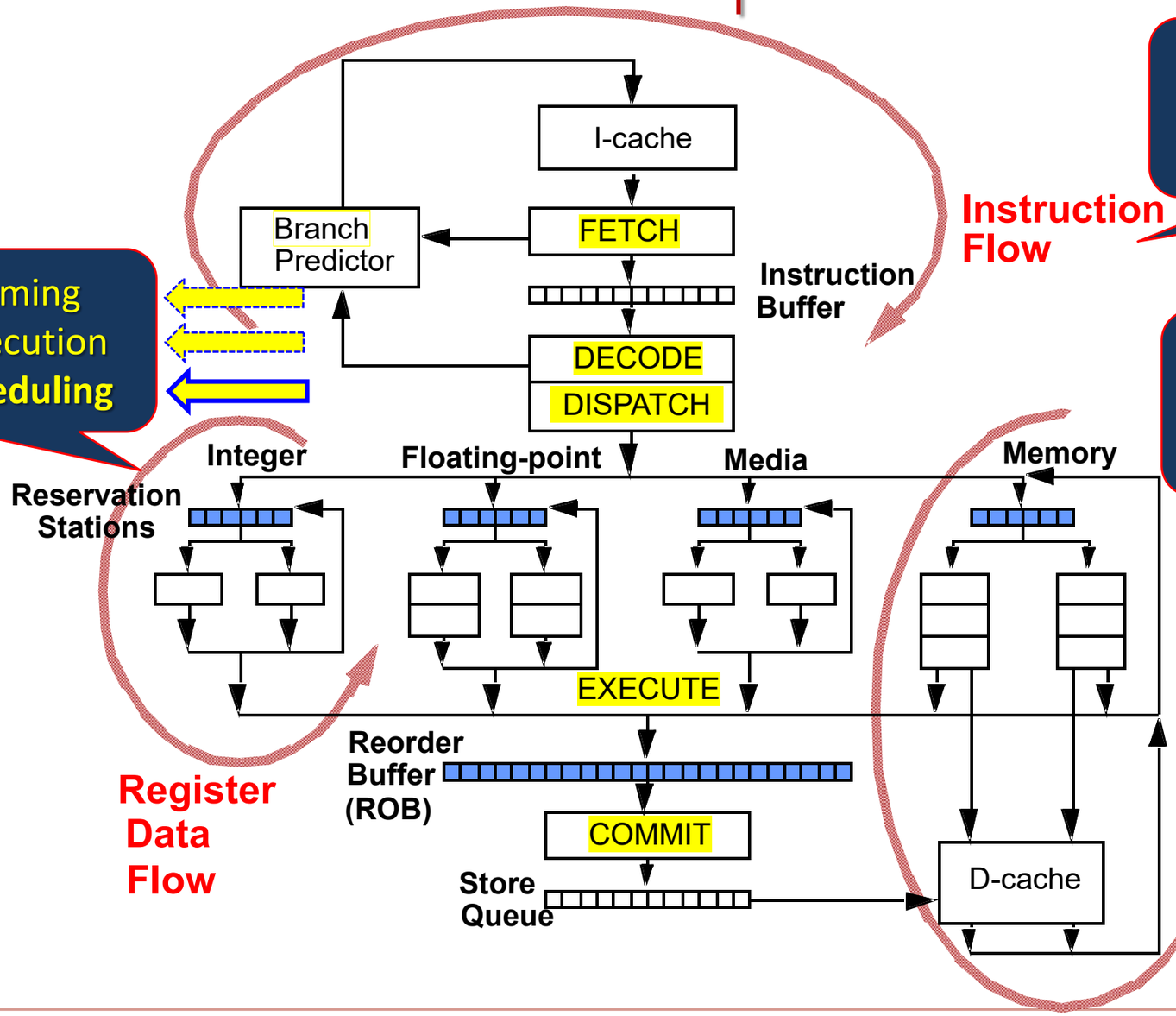
18-740: Modern Computer Architecture & Design

Lecture 6:
"Dynamic Instruction Scheduling Techniques"

A. Dynamic Instruction Scheduling (IQ/RS)
B. Out-of-Order Instruction Execution (ROB)
C. Intel Pentium Pro (P6) Legacy (1995-2015)

Electrical & Computer ENGINEERING

Three Flow Paths of Superscalar Processors



- Dynamic Branch Prediction
- Wide Instruction Fetching
- Dynamic Micro/Macro OPS

- Register Renaming
- Data Flow Execution
- **Dynamic Scheduling**

- Load Bypassing & Forwarding
- Speculative Memory Disamb.
- Dynamic Memory Prefetching

A. Dynamic Instruction Scheduling (IQ/RS)

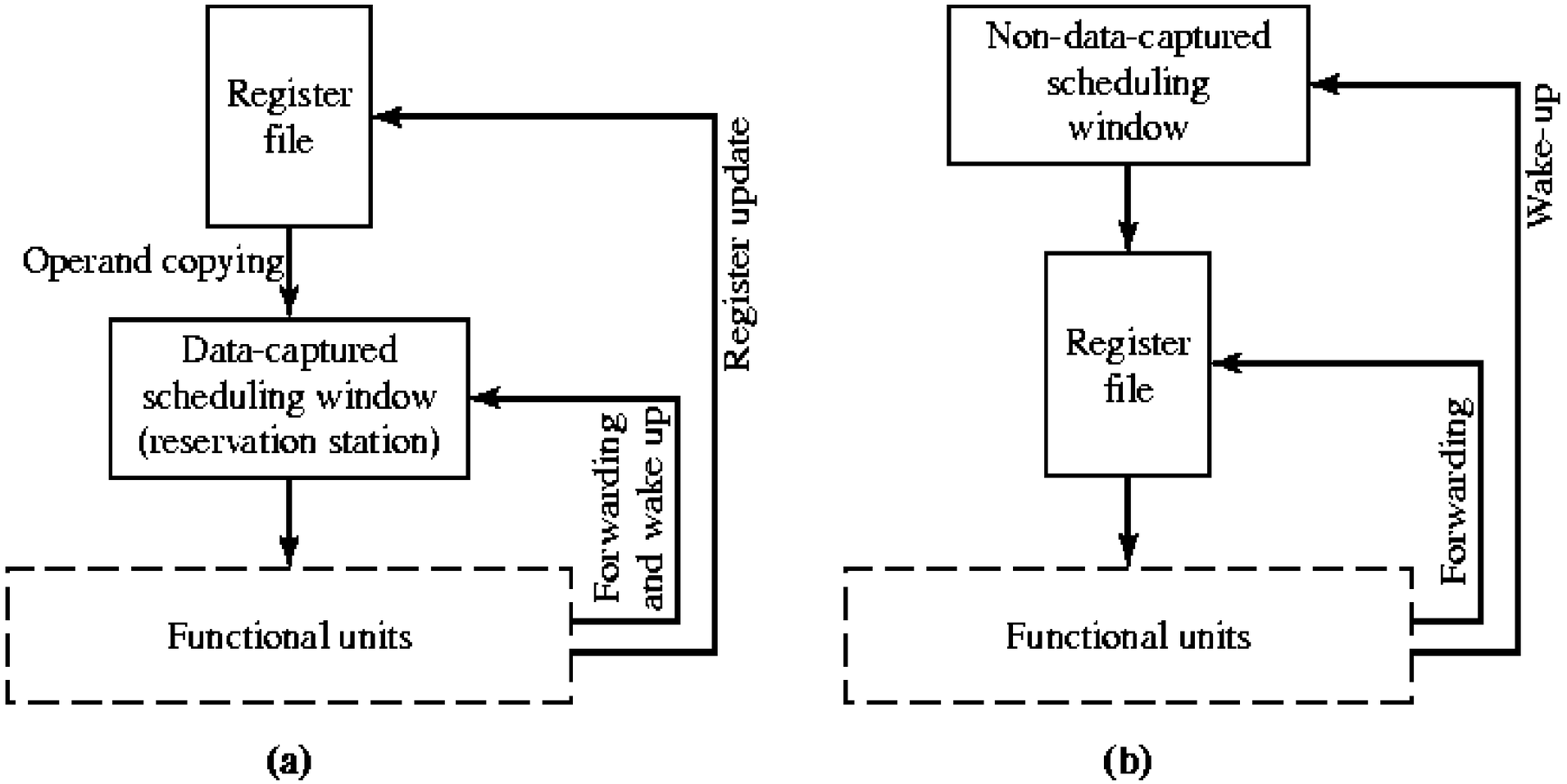
0. Result Forwarding
1. Wake Up
2. Select
3. Issue

Instruction Queue Implementation Alternatives

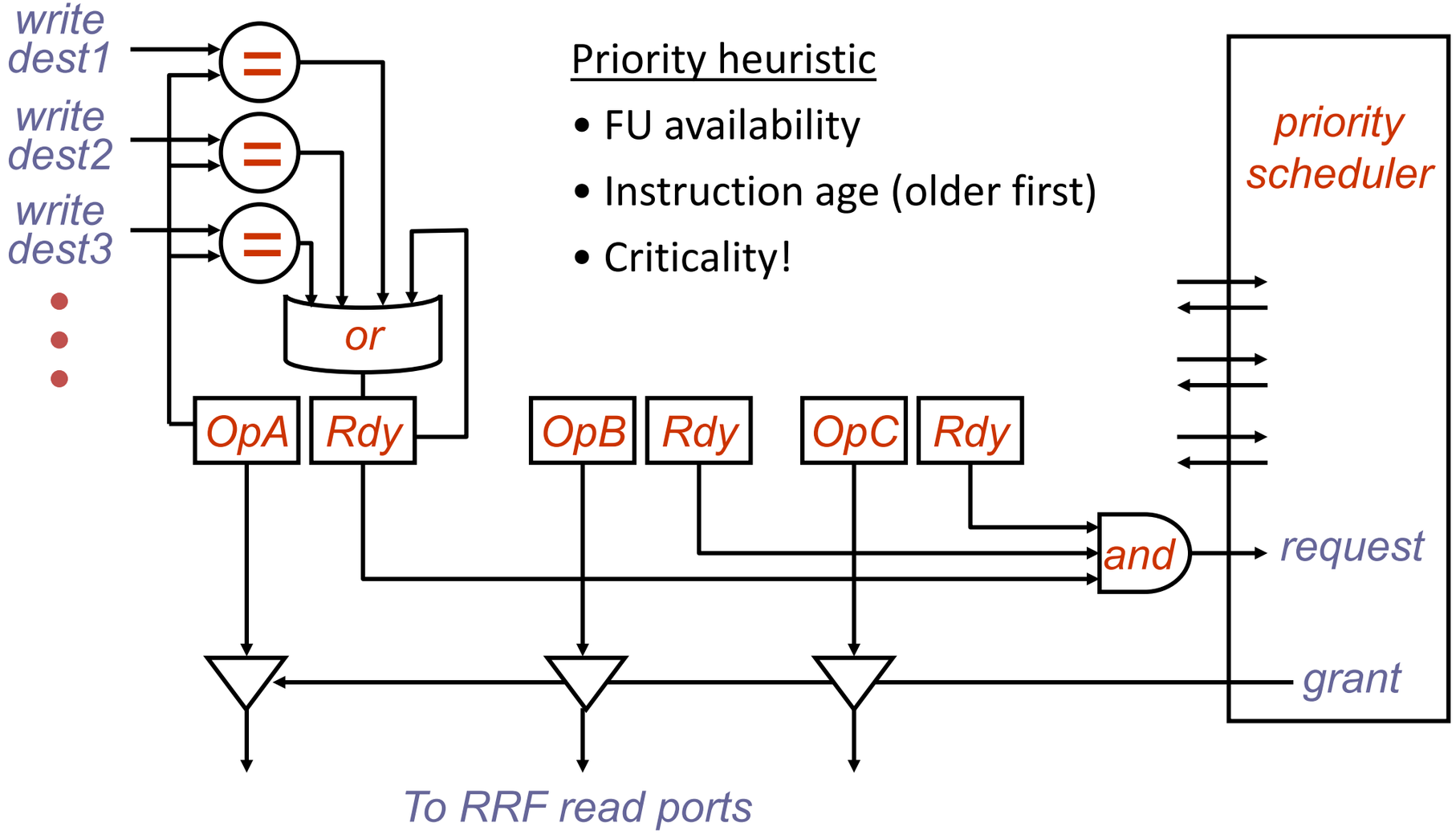
- Single vs. multiple buffers (trade-offs?)
  - Single centralized window
  - Single centralized window with static alignment for different FUs
  - Separate INT – FP – LSU windows
  - Separate buffers for each FU type
    - Reservation Stations (RS)
- Management policies to keep in mind
  - Random access or FIFO
    - In-order vs. out-of-order within each queue
  - Age-prioritized or criticality based
  - Value vs. Tag only
  - When to de-allocate
    - Reservation Stations for Ld/St units are more complicated

Dynamic Instruction Scheduler Options

With Data Capture vs. Without Data Capture



Instruction Queue Scheduling



Priority heuristic

- FU availability
- Instruction age (older first)
- Criticality!

0. Result Forwarding

- Common data bus: used to broadcast results of FUs
- Broadcast destinations
  - ARF or RRF or ROB, depending on the renaming scheme
  - Instruction window
    - May need result or tag for the result
- Number of CDBs
  - Best case, 1 per functional unit
  - Can have less, but now we may have structural hazard due to contention
- Notes:
  - CDBs can be slow as the routing goes across large chip area
  - Broadcast tag early to help set up data forwarding

1. Wake-up Latency

- Assume a result becomes available in cycle i
  - When can you start executing an instruction that is waiting for it?
- Ideal solution: in cycle i+1
  - Back to back executing, just like with 5-stage pipeline
  - Requirement: the following will have to work in one cycle
    - Distribute result tag to the window and detect that instruction becomes ready
    - Select instruction for execution and forward its info/operands to FU
  - May stress clock cycle in wide processors
- Alternative: split **wake-up** and **select** in separate cycles
  - Simpler hardware, faster clock cycle
  - Lower IPC (dependencies cost one extra cycle)

2. Priority/Select Logic Issues

- Selection is easier if priority depends on instruction location
  - Older instructions are at the bottom of window and receive priority
- This creates an issue of compacting/collapsing
  - As instructions depart, compress remaining entries towards the bottom
  - Younger instructions will be inserted towards the top (lower priority)
- Compacting the window can be quite complex
  - Its complexity can affect performance (clock frequency)
  - Often implemented in some restricted form
    - E.g. split window into multiple groups, allow compaction of groups
    - Trade-off between window utilization and compaction simplicity

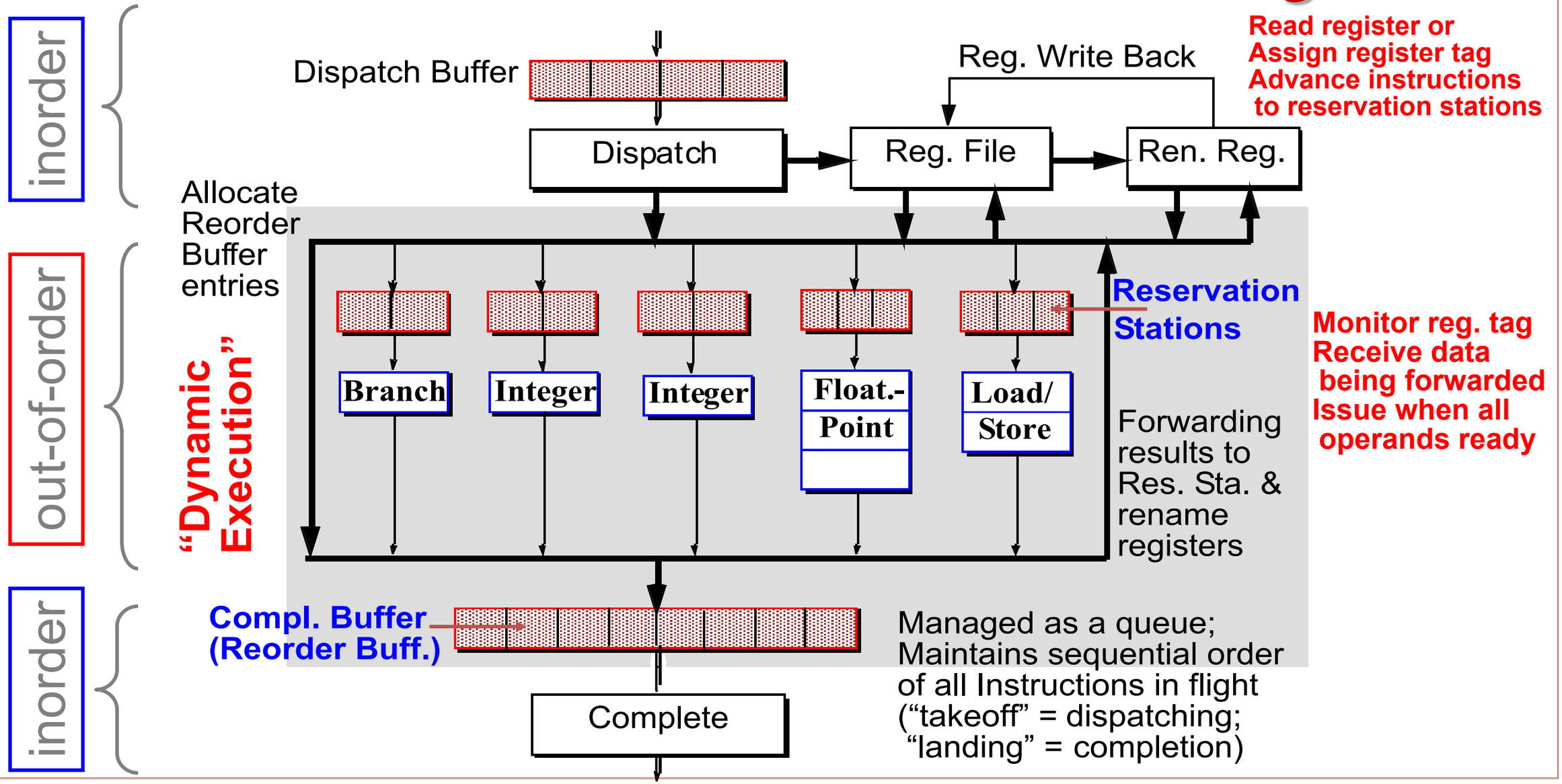
Dynamic Scheduling Implementation Cost

- To support N-way dispatch into IW per cycle
  - Nx2 simultaneous lookups into the rename map (or associative search)
  - N simultaneous write ports into the IW and the ROB
- To support N-way issue per cycle (assuming read at issue)
  - 1 prioritized associative lookup of N entries
  - N read ports into the IW
  - Nx2 read ports into the RF
- To support N-way finish per cycle
  - N write ports into the RRF or the ROB
  - Nx2 associative lookup and write in the IW or RSs
- To support N-way retire per cycle
  - N read ports in the ROB
  - N ports into the ARF (potentially)

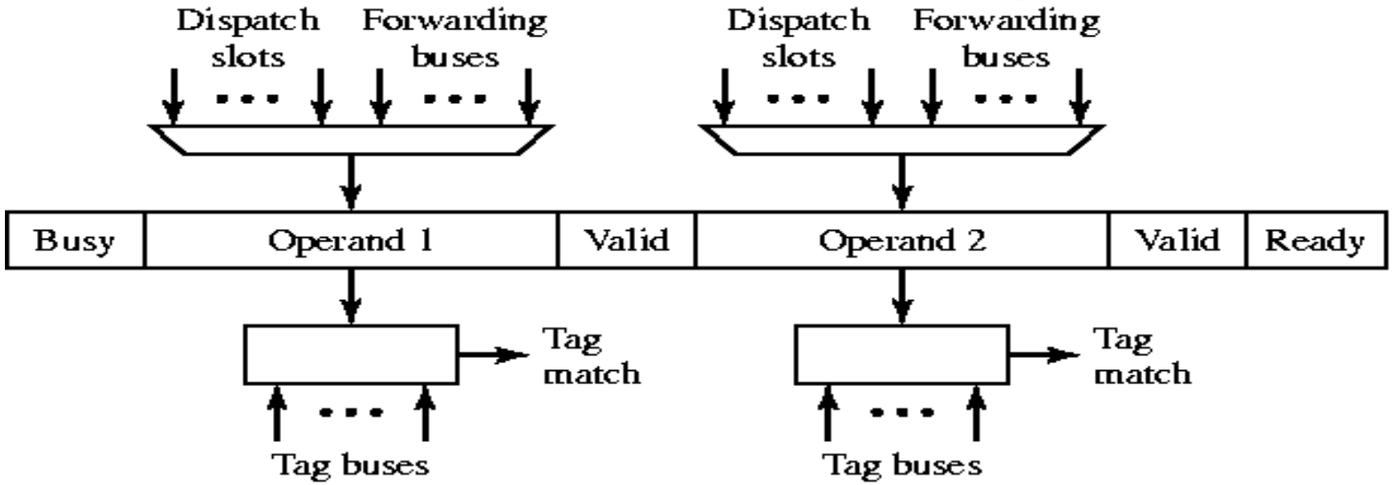
B. Out-of-Order Instruction Execution (ROB)



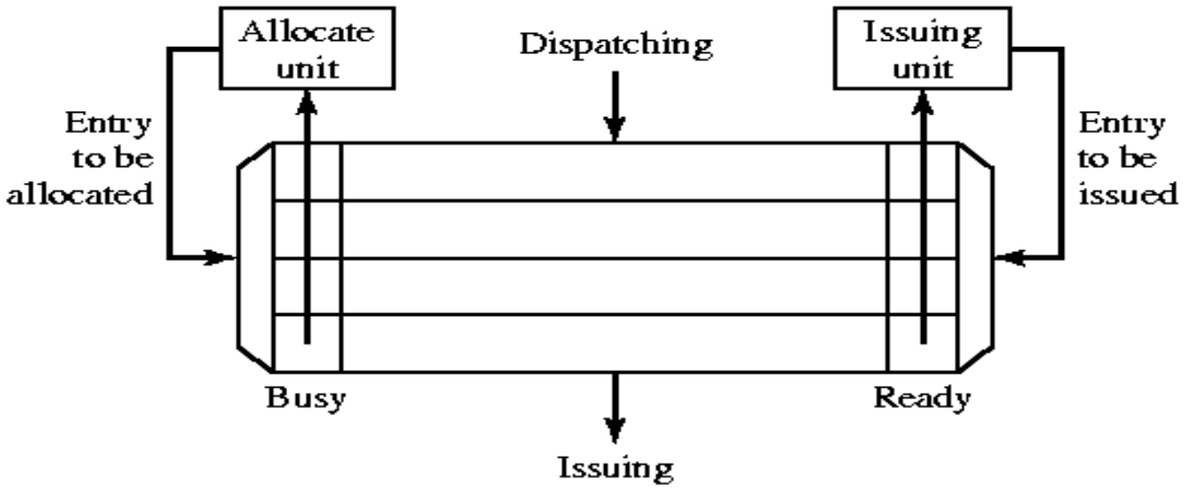
Elements of Modern Micro-Dataflow Engine



Reservation Station Implementation



(a)



(b)

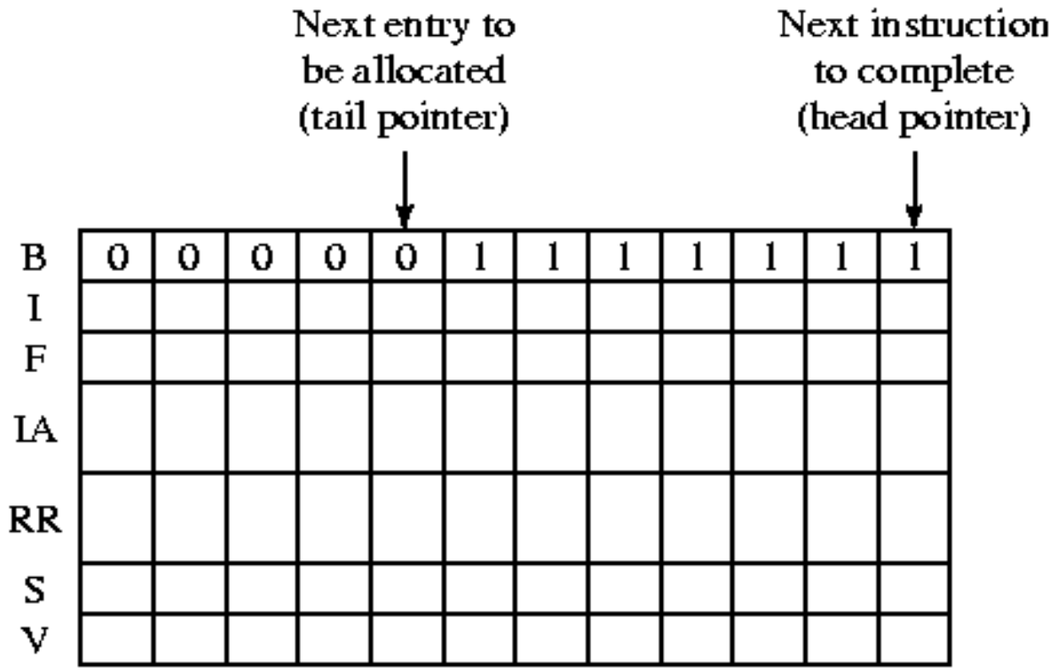
+ info for executing instruction (opcode, ROB entry, RRF entry…)

- Reservation Stations: distributed vs. centralized
  - Wakeup: benefit to partition across data types
  - Select: much easier with partitioned scheme
    - Select 1 of n/4 vs. 4 of n

Reorder Buffer Implementation

Busy	Issued	Finished	Instruction address	Rename register	Speculative	Valid

(a)



(b)

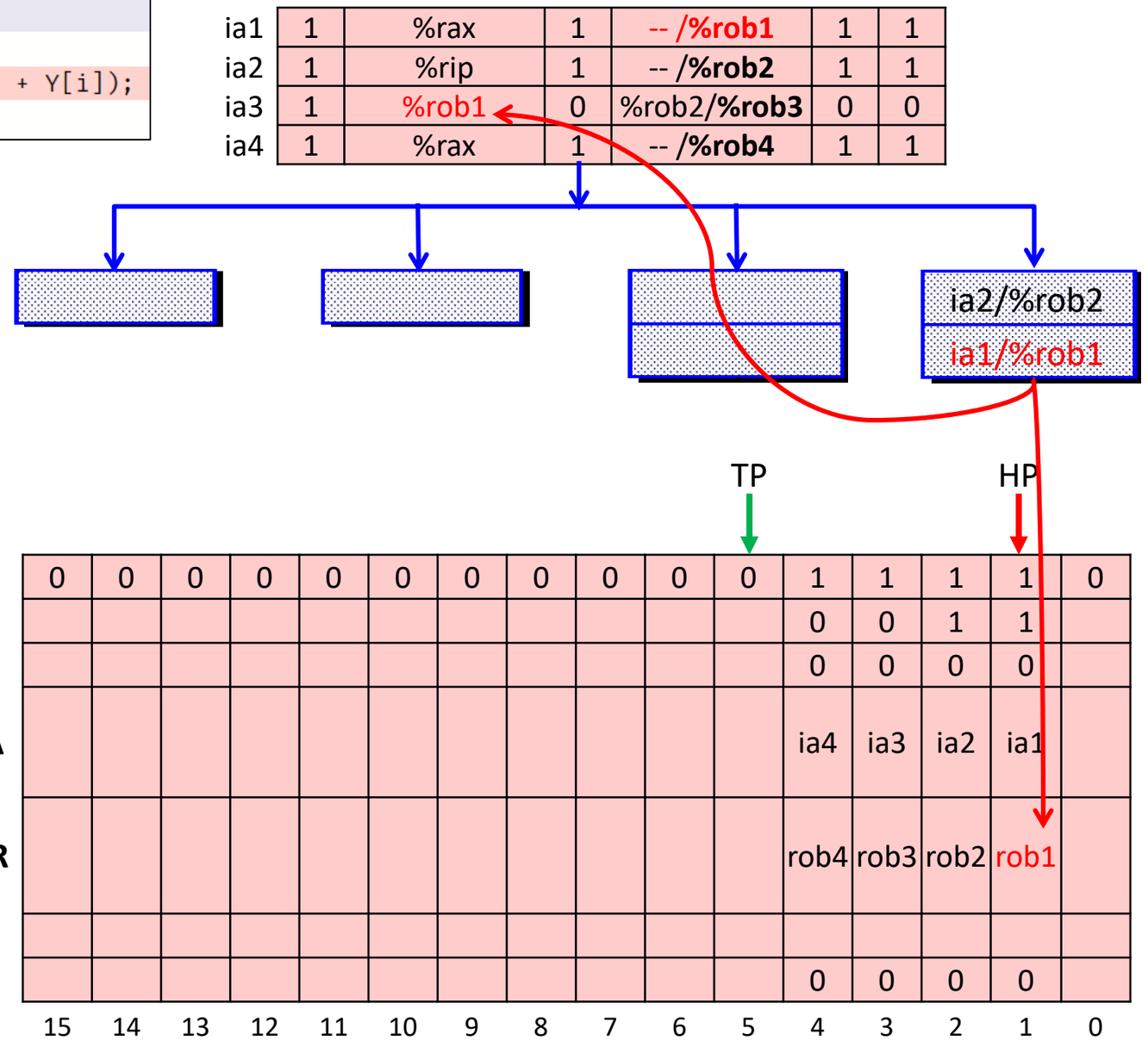
- Reorder Buffer
  - "Bookkeeping"
  - Can be instruction-grained, or block-grained (4-5 ops)

Cycle: 001

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

Four instructions (ia1, ia2, ia3, ia4) have been dispatched to RS with four corresponding entries allocated in ROB. ia1 and ia2 have been issued into LS FU. Next cycle, ia1 will finish and broadcast its result using tag “%rob1” to ia3.



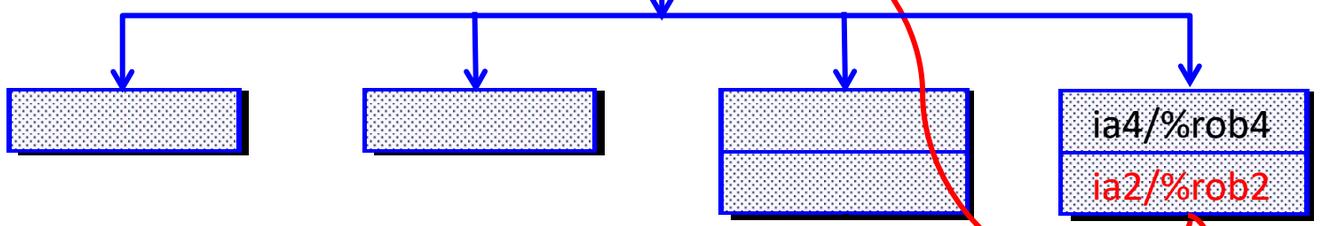
Cycle: 002

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

ia1 completes and is next to leave ROB.
Its RS entry has been reallocated to ia5.
ia4 is issued into LS FU.
Next cycle, ia2 will broadcast its results using tag “%rob2” to ia3.

ia5	1	%rob3	0	%rob4/**%rob5**	1	1
ia2	1	%rip	1	-- /**%rob2**	1	1
ia3	1	%rob1	1	%rob2/**%rob3**	0	0
ia4	1	%rax	1	-- /**%rob4**	1	1



TP HP

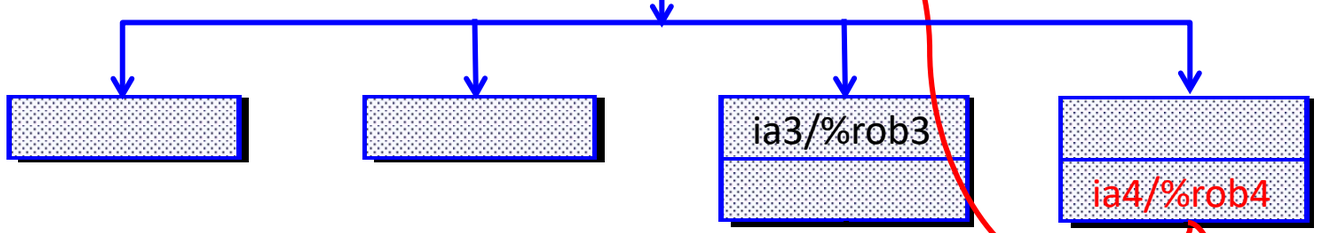
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0
I											0	1	0	1	1	
F											0	0	0	0	1	
IA											ia5	ia4	ia3	ia2	ia1	
RR											Rob5	rob4	rob3	rob2	rob1	
S																
V											0	0	0	0	1	

Cycle: 003

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

ia5	1	%rob3	0	%rob4/**%rob5**	0	0
ia6	1	%rob5	0	--	1	0
ia3	1	%rob1	1	%rob2/**%rob3**	1	1
ia4	1	%rax	1	-- /**%rob4**	1	1



TP HP

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0
I										0	0	1	1	1		
F										0	0	0	0	1		
IA										ia6	ia5	ia4	ia3	ia2		
RR											rob5	rob4	rob3	rob2		
S																
V										0	0	0	0	1		

ia2 completes and is ready to leave ROB.
ia3 is now ready and is issued to MULT FU.
Next cycle, ia4 will forward result to ia5 in RS using tag “%rob4”.

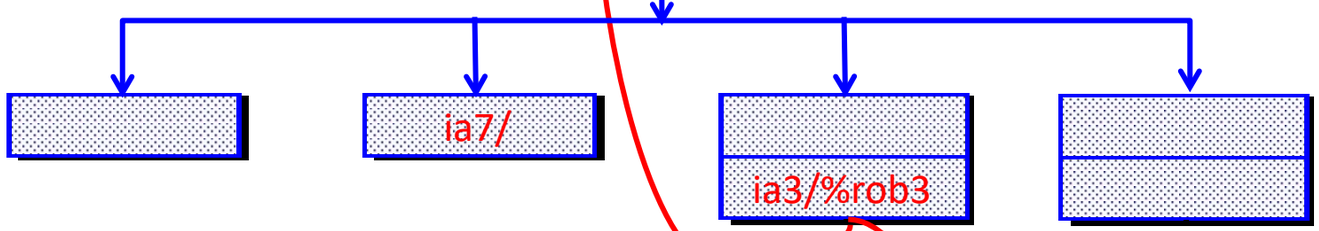
Cycle: 004

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

ia4 finishes but must wait for ia3 before it can leave ROB.
ia7 has been dispatched to RS and allocated entry in ROB, and is issued.
Next cycle, ia3 will forward result to ia5 using the tag "%rob3".

ia5	1	%rob3	0	%rob4/**%rob5**	1	0
ia6	1	%rob5	0	--	1	0
ia3	1	%rob1	1	%rob2/**%rob3**	1	1
ia7	1	$1	1	%rbp	1	1



TP HP

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0
I									1	0	0	1	1			
F									0	0	0	1	0			
IA									ia7	ia6	ia5	ia4	ia3			
RR											rob5	rob4	rob3			
S																
V									0	0	0	1	0			

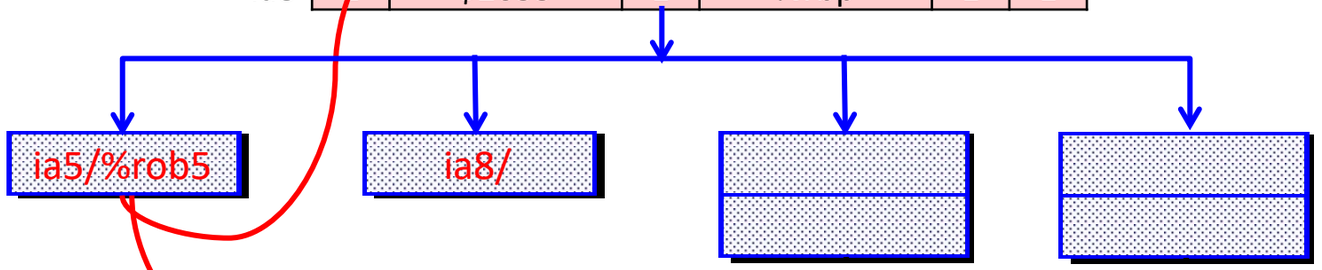
Cycle: 005

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

ia5 is now ready and is issued to FU.
Next cycle, ia5 will forward result to ia6 using the tag "%rob5".
ia7 finishes, its RS deallocated, but must wait in ROB. ia3 and ia4 ready to retire.
ia8 is dispatched to RS and issued to FU.

ia5	1	%rob3	1	%rob4/**%rob5**	1	1
ia6	1	%rob5	0	--	1	0
ia3	1	%rob1	1	%rob2/**%rob3**	1	1
ia8	1	$2055	1	%rbp	1	1



TP HP

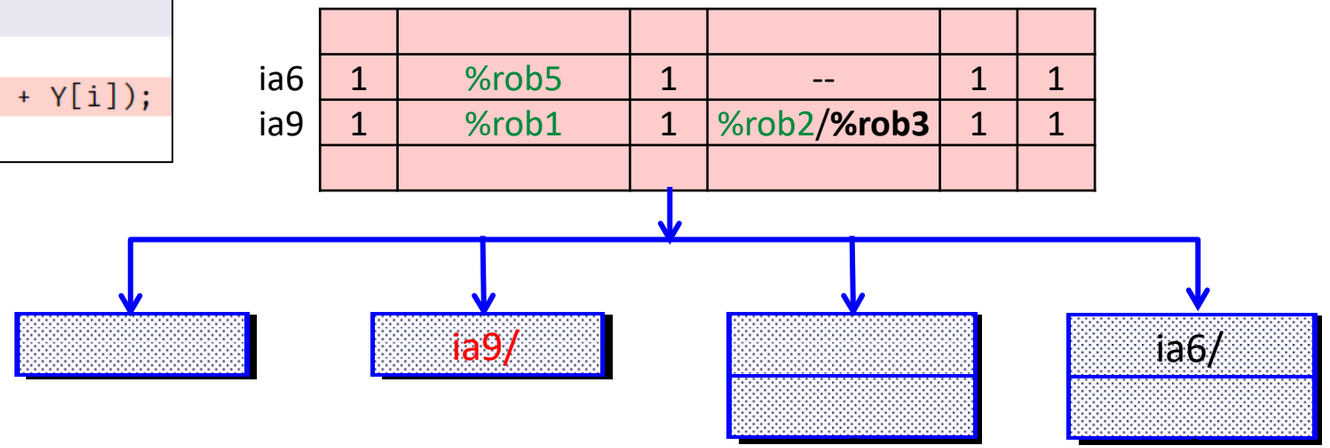
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B	0	0	0	0	0	0	0	1	1	1	1	1	1	0	0	0
I								1	1	0	1	1	1			
F								0	1	0	0	1	1			
IA								ia8	ia7	ia6	ia5	ia4	ia3			
RR											rob5	rob4	rob3			
S																
V								0	1	0	0	1	1			

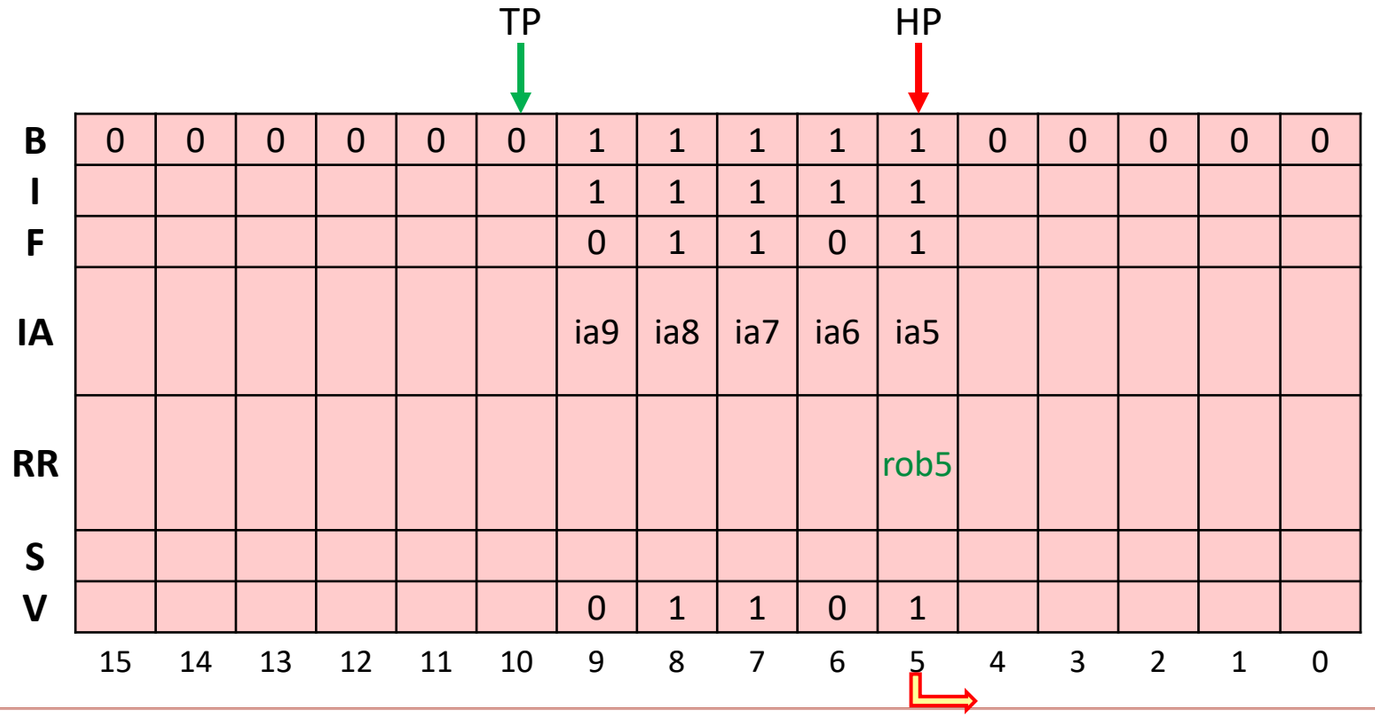
Cycle: 006

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

ia6 is now ready and is issued to LS FU.
ia9 is dispatched to RS and issued into FU.
ia5 finishes, its RS deallocated, and is ready to leave ROB.
ia8 finishes, its RS deallocated, but must wait in ROB.



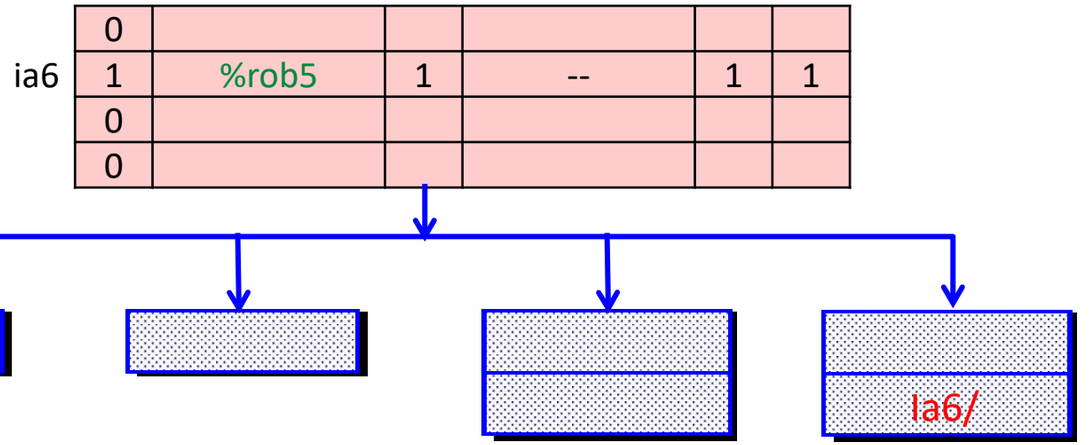


Cycle: 007

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

ia9 finishes, its RS deallocated, but must wait in ROB.
ia6 is finishing and will be complete next cycle and be ready to leave ROB.



TP HP

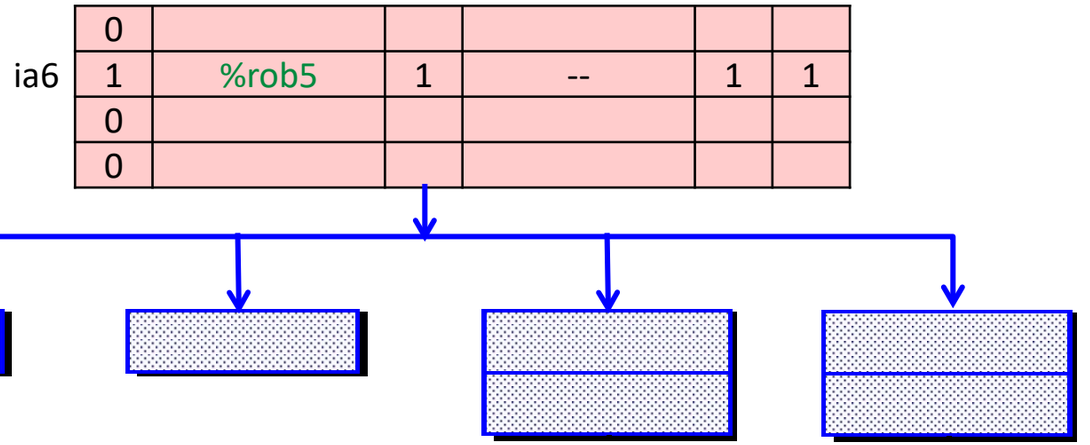
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0
I							1	1	1	1						
F							1	1	1	0						
IA							ia9	ia8	ia7	ia6						
RR																
S																
V							1	1	1	0						

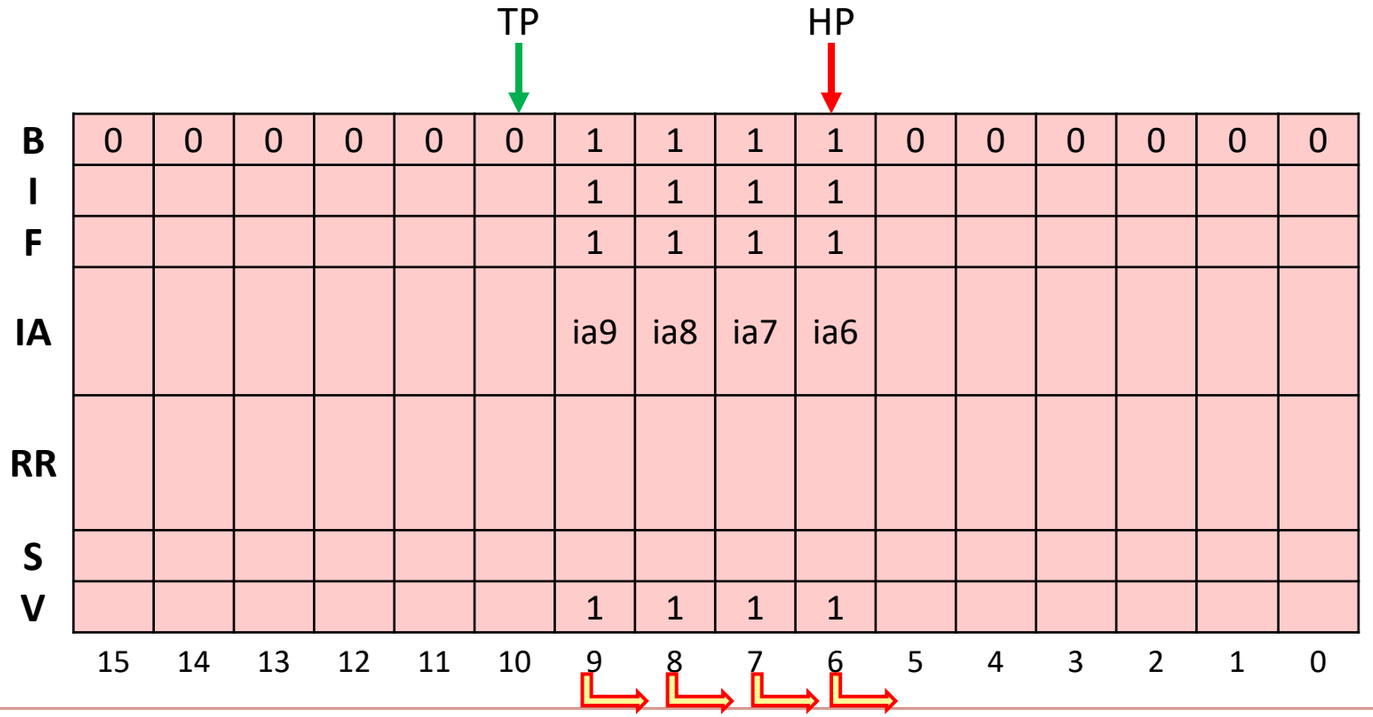
Cycle: 008

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```

i6 completes and all four remaining instructions will leave ROB next cycle.

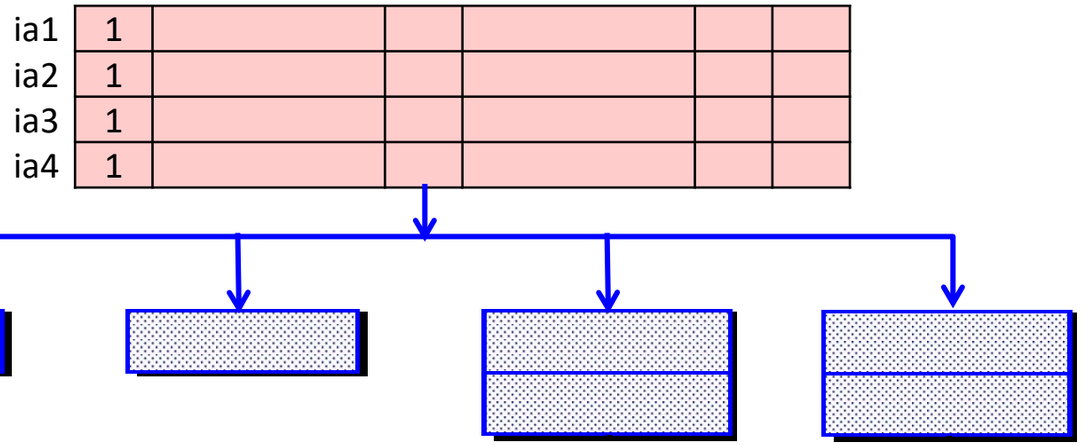


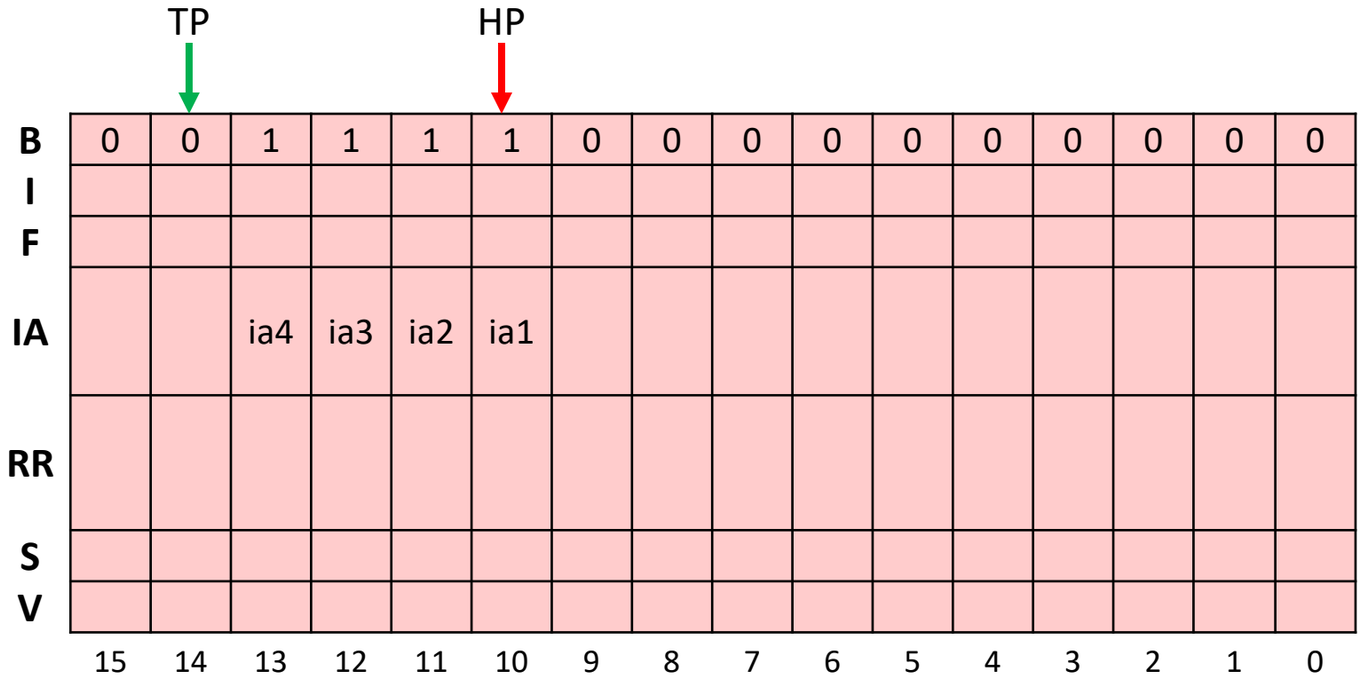


Cycle: 009

```
// Implment DAXPY here
for(i = 0; i < N; i++)
{
    Y[i] = (alpha * X[i] + Y[i]);
}
```

```
        movl $0, -4(%rbp)
        jmp .L2
.L3:
ia1     movsd X(,%rax,8), %xmm1
ia2     movsd alpha(%rip), %xmm0
ia3     mulsd %xmm1, %xmm0
ia4     movsd Y(,%rax,8), %xmm1
ia5     addsd %xmm1, %xmm0
ia6     movsd %xmm0, Y(,%rax,8)
ia7     addl $1, -4(%rbp)
.L2:
ia8     cmpl $2055, -4(%rbp)
ia9     jle .L3
```





ia9 is executed and branches to top of the loop and starts next iteration by dispatching ia1, ia2, ia3, and ia4 to RS and ROB. The same process is repeated.

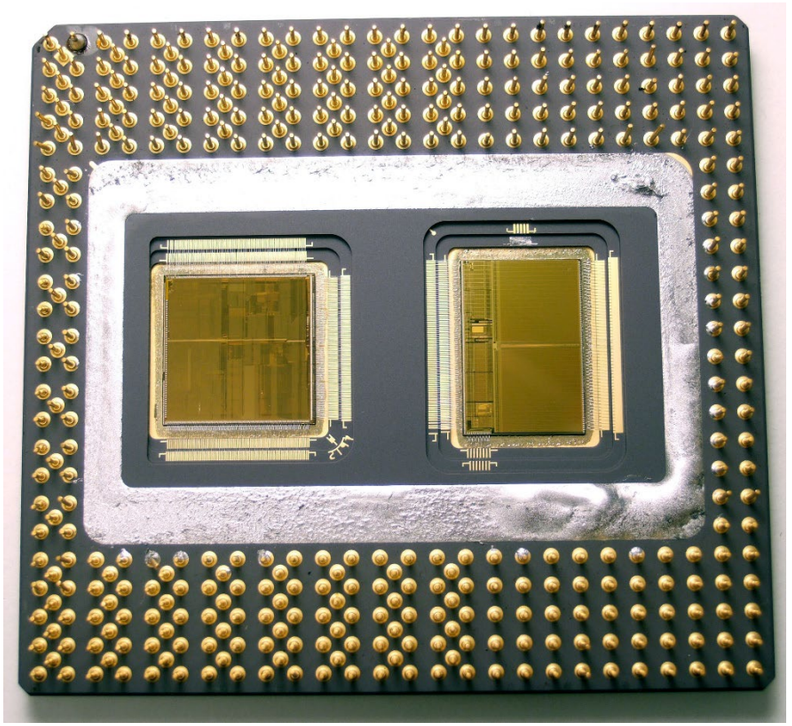
C. Intel Pentium Pro (P6) Legacy (1995…2015)

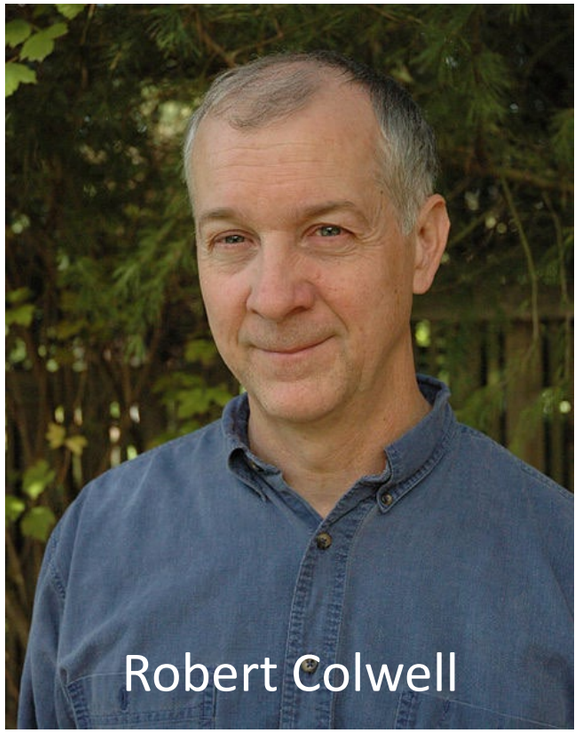
http://en.Wikipedia.org/wiki/Pentium_Pro

Electrical & Computer ENGINEERING

Intel Pentium Pro (P6) Case Study

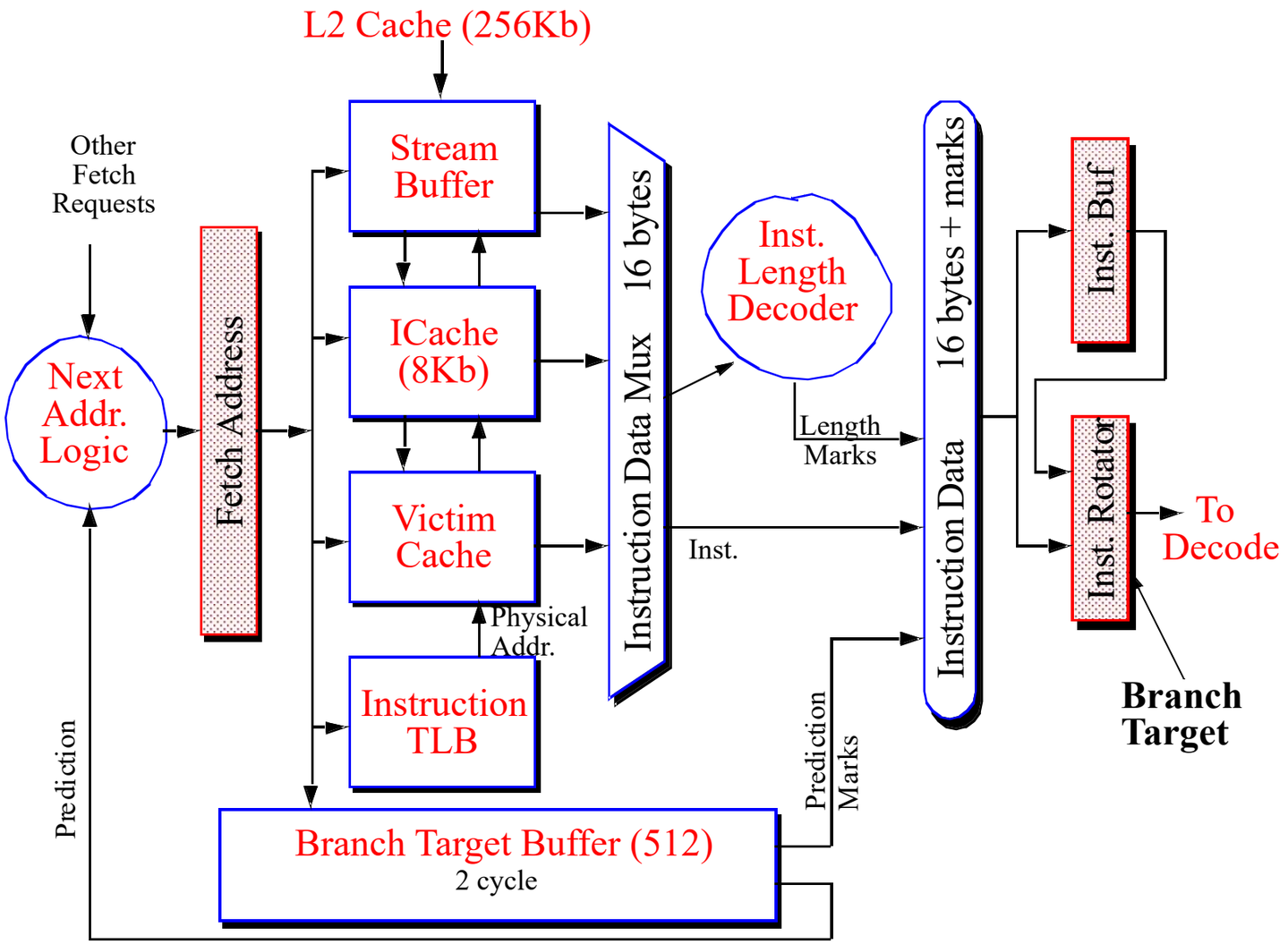
- **Microarchitecture**
  - Order-3 Superscalar
  - Out-of-Order execution
  - Speculative execution
  - In-order completion
- **Design Methodology**
- **Retrospective**



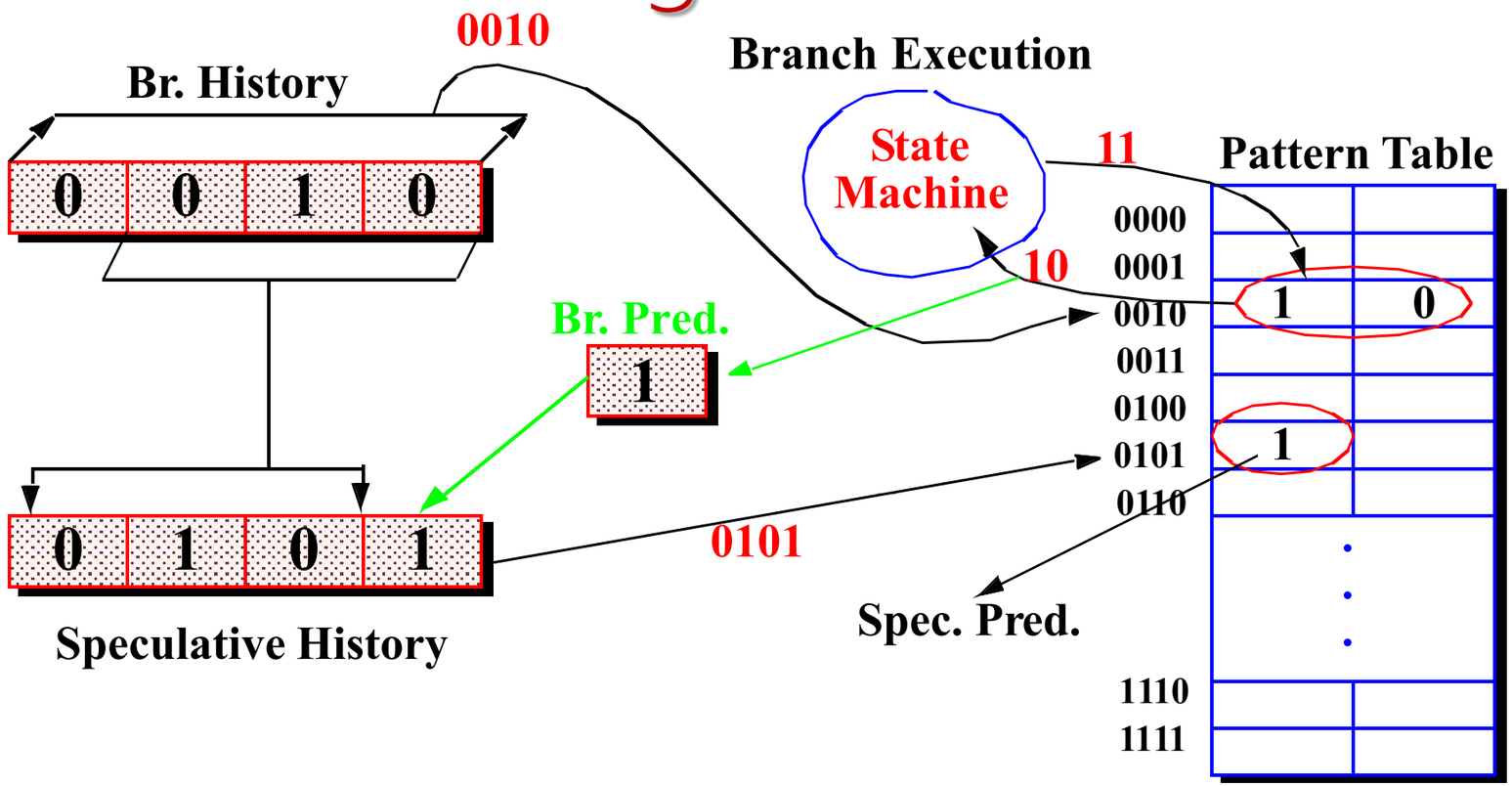


Introduced in 1995

Instruction Fetch

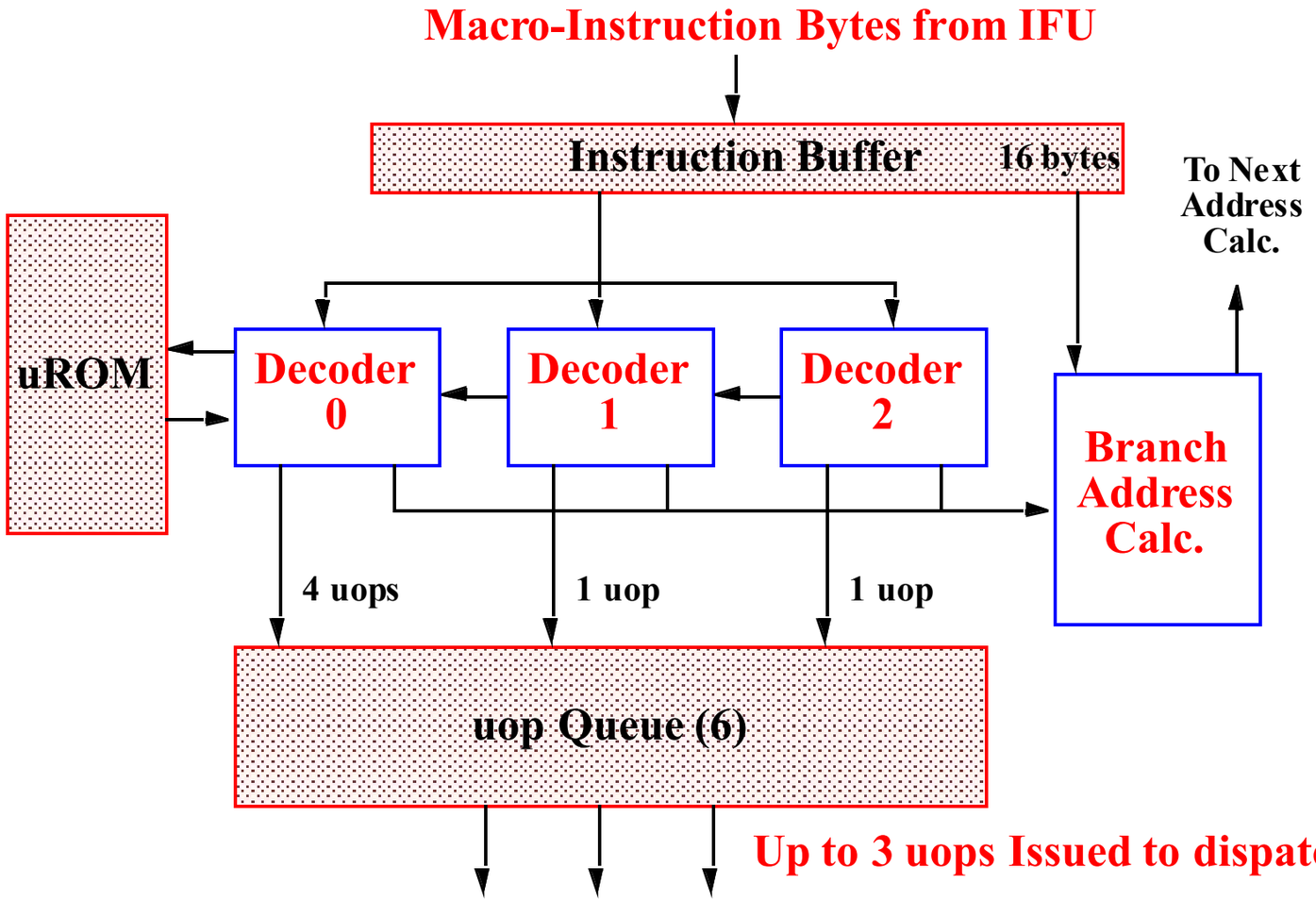


Branch Prediction Algorithm



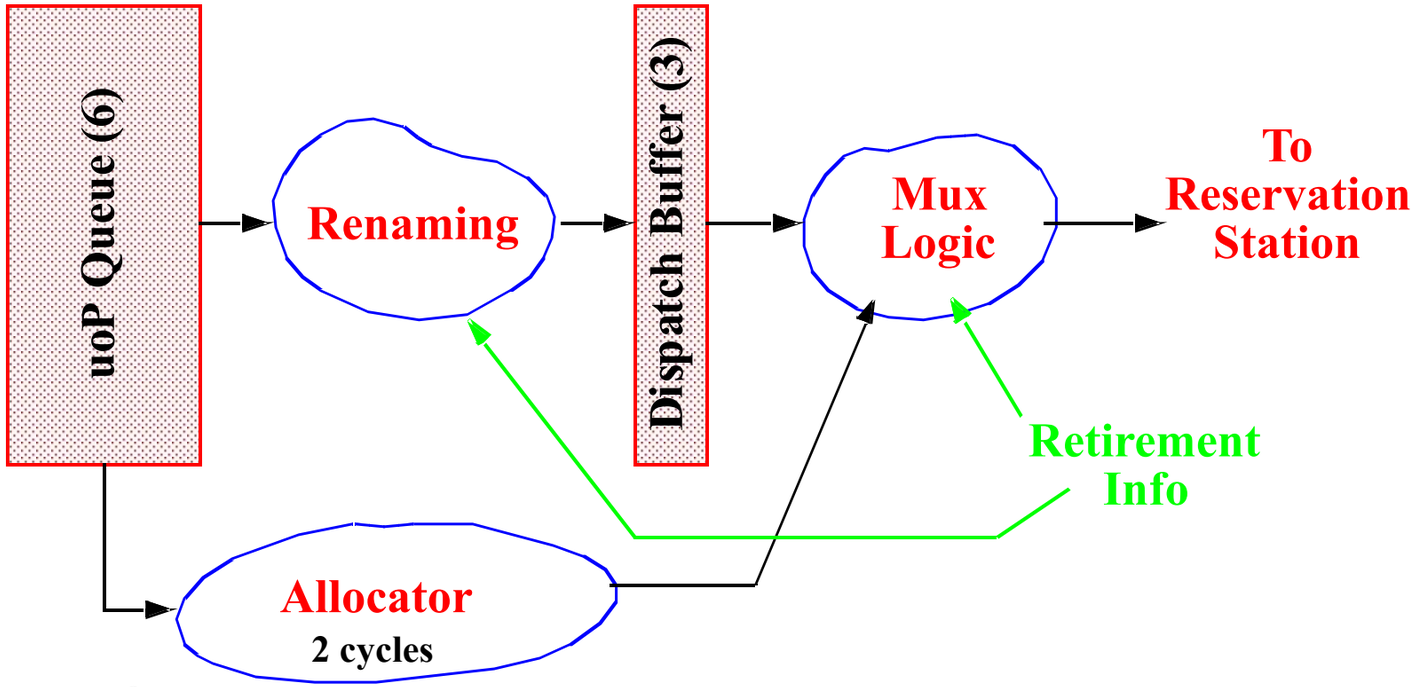
- Current prediction updates the speculative history prior to the next instance of the branch instruction
- Branch History Register (BHR) is updated during branch execution
- Branch recovery flushes front-end and drains the execution core
- Branch mis-prediction resets the speculative branch history state to match BHR

Instruction Decode



- Instruction Buffer contains up to 16 instructions, which must be decoded and queued before the instruction buffer is re-filled
- Macro-instructions must shift from decoder 2 to decoder 1 to decoder 0

Instruction Dispatch



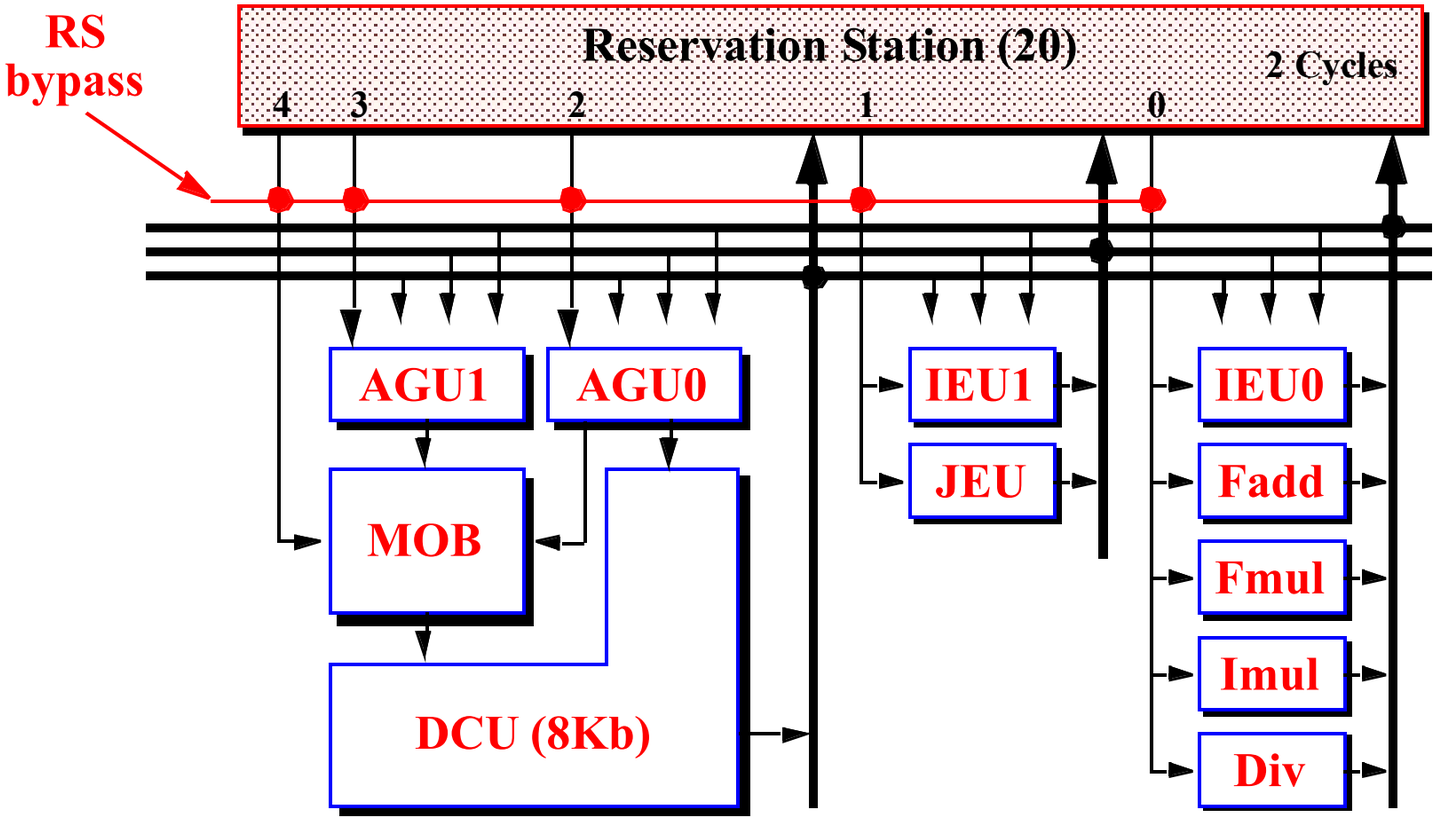
Register Renaming

Allocation requirements

- **"3-or-none" Reorder buffer entries**
- **Reservation station entry**
- **Load buffer or store buffer entry**

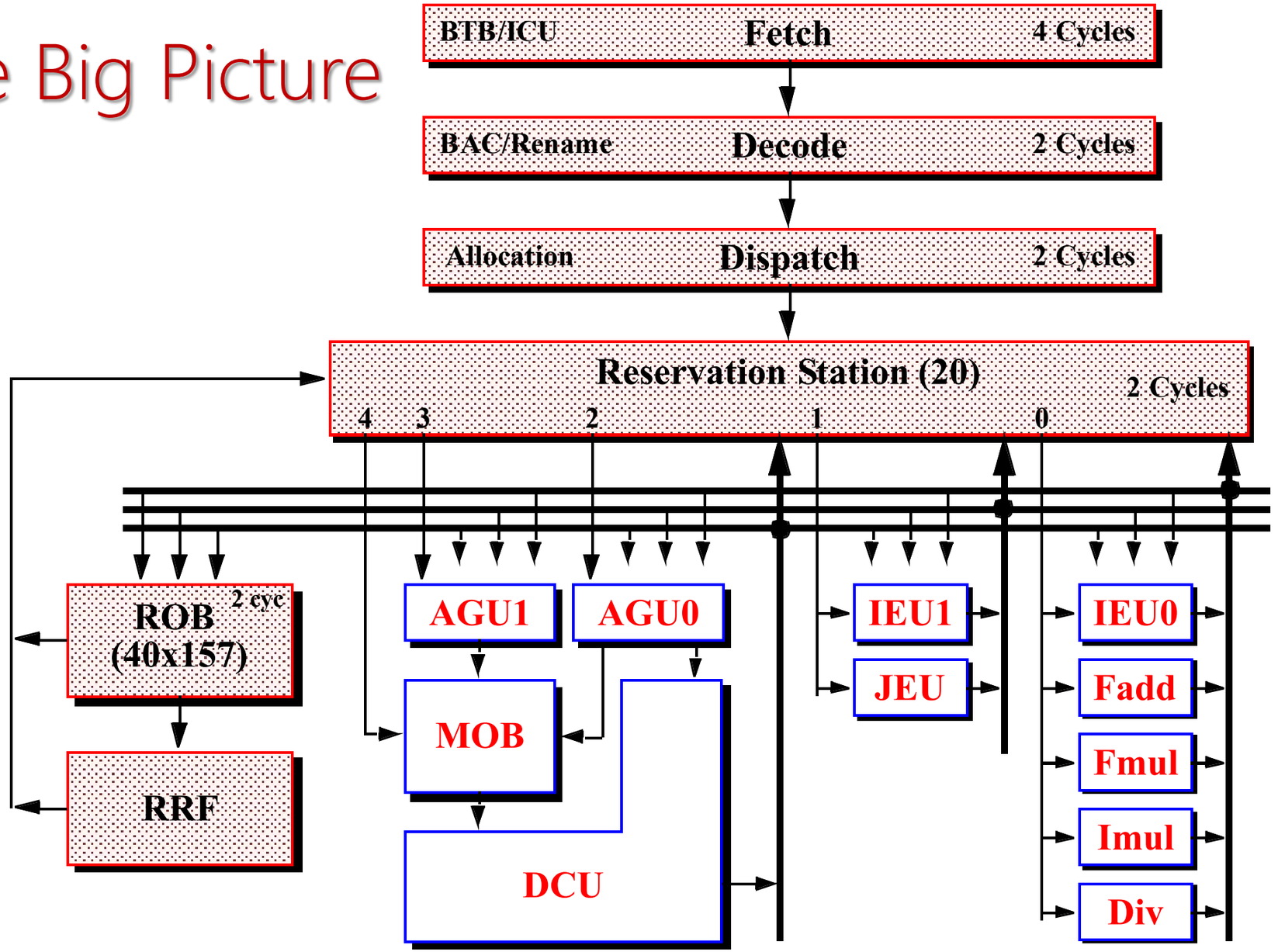
Dispatch buffer *"probably"* dispatches all 3 uops before re-fill

Out-of-Order Execution Engine



- In-order branch issue and execution
- In-order load/store issue to address generation units
- Instruction execution and result bus scheduling
- Is the reservation station *"truly"* centralized & what is *"binding"*?

P6 – The Big Picture



Retrospective on the Intel P6 (Pentium Pro)

- Most commercially successful microarchitecture in history
- P6-Based Evolution (1995 – 2015)
  - Pentium II/III, Xeon, etc.
    - Derivatives with on-chip L2, ISA extensions, etc.
  - Replaced by Pentium 4 as flagship in 2001
    - High frequency, deep pipeline, extreme speculation (last design in 2005)
  - Resurfaced as Pentium M in 2003
    - Initially a response to Transmeta in laptop market
    - Pentium 4 derivative (90nm Prescott) delayed, slow, hot
  - Core Duo, Core 2 Duo, Core i7 (replaced Pentium 4 around 2006)

Microarchitectural Updates

- Pentium M (Banias), Core Duo (Yonah)
  - Micro-op fusion (also in AMD K7/K8)
    - Multiple uops in one: (add eax,[mem] => ld/alu), sta/std
    - These uops decode/dispatch/commit once, issue twice
  - Better branch prediction
    - Loop count predictor
    - Indirect branch predictor
  - Slightly deeper pipeline (12 stages)
    - Extra decode stage for micro-op fusion
    - Extra stage between issue and execute (for RS/PLRAM read)
  - Data-capture reservation station (payload RAM)
    - Clock gated for 32 (int) , 64 (fp), and 128 (SSE) operands

Microarchitectural Updates

- Core 2 Duo (Merom)
  - 64-bit ISA from AMD K8
  - Macro-op fusion
    - Merge uops from two x86 ops
    - E.g. cmp, jne => cmpjne
  - 4-wide decoder (Complex + 3x Simple)
    - Peak x86 decode throughput is 5 due to macro-op fusion
  - Loop buffer
    - Loops that fit in 18-entry instruction queue avoid fetch/decode overhead
  - Even deeper pipeline (14 stages)
  - Larger reservation station (32), instruction window (96)
  - Memory dependency prediction

Microarchitectural Updates

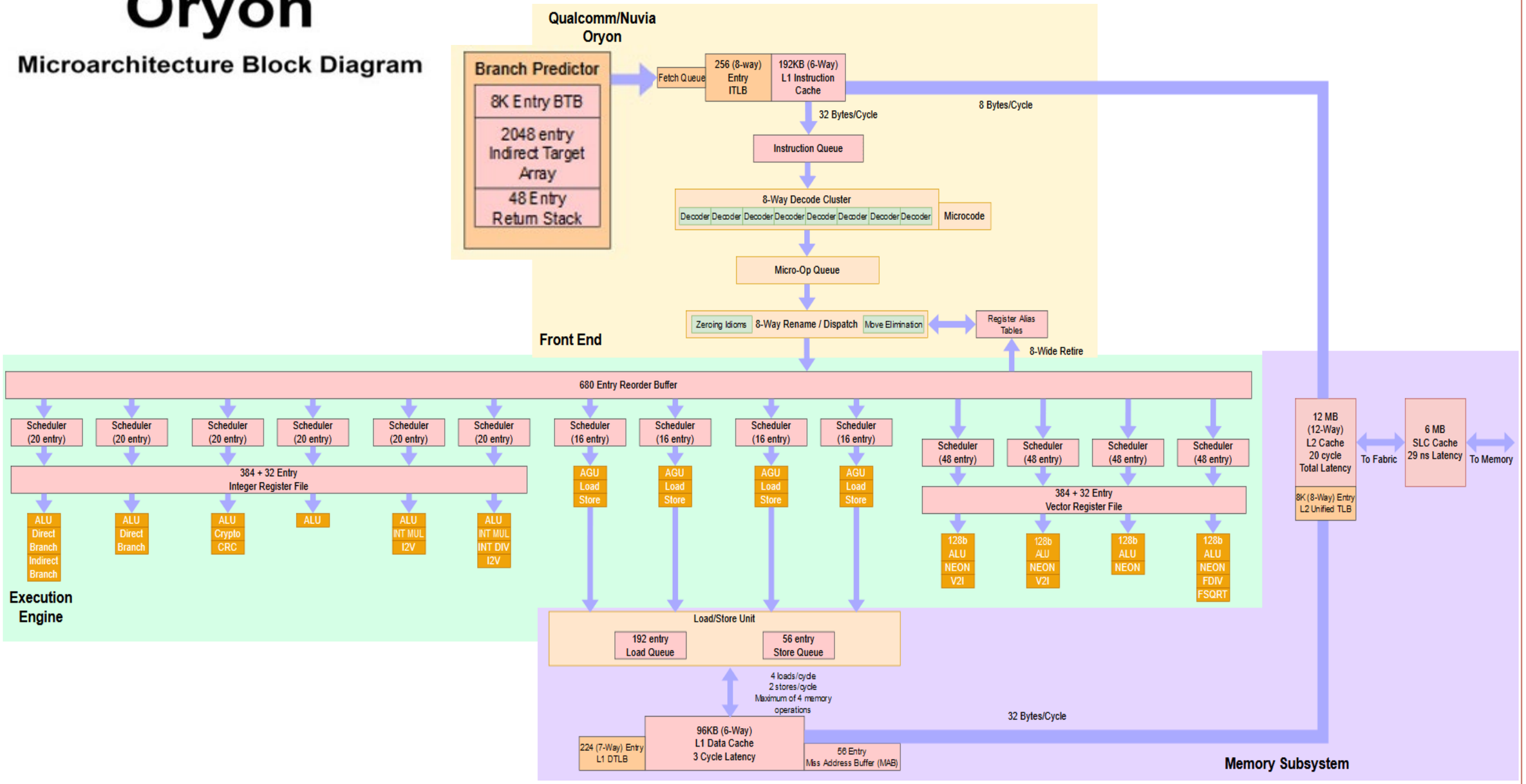
- Nehalem (Core i7/i5/i3) (2008-)
  - RS size 36, ROB 128
  - Loop cache up to 28 uops
  - L2 branch predictor
  - L2 TLB
  - I$ and D$ now 32K, L2 back to 256K, inclusive L3 up to 8M
  - Simultaneous multithreading
  - RAS now renamed (repaired)
  - 6 issue, 48 load buffers, 32 store buffers
  - New system interface (QPI) – finally dropped front-side bus
  - Integrated memory controller (up to 3 channels)
  - New STTNI instructions for string/text handling

Microarchitectural Updates

- Sandy Bridge (2nd generation Core i7) (2011-) Ivy Bridge (2012-)
  - On-chip integrated graphics (GPU)
  - Decoded uop cache up to 1.5K uops, handles loops, but more general
  - 54-entry RS, 168-entry ROB
  - Physical register file: 144 FP, 160 integer
  - 256-bit AVX units: 8 DPFLOP/cycle, 16 SPFLOP/cycle
  - 2 general AGUs enable 2 ld/cycle, 2 st/cycle or any combination, 2x128-bit load path from L1 D$

Oryon

Microarchitecture Block Diagram

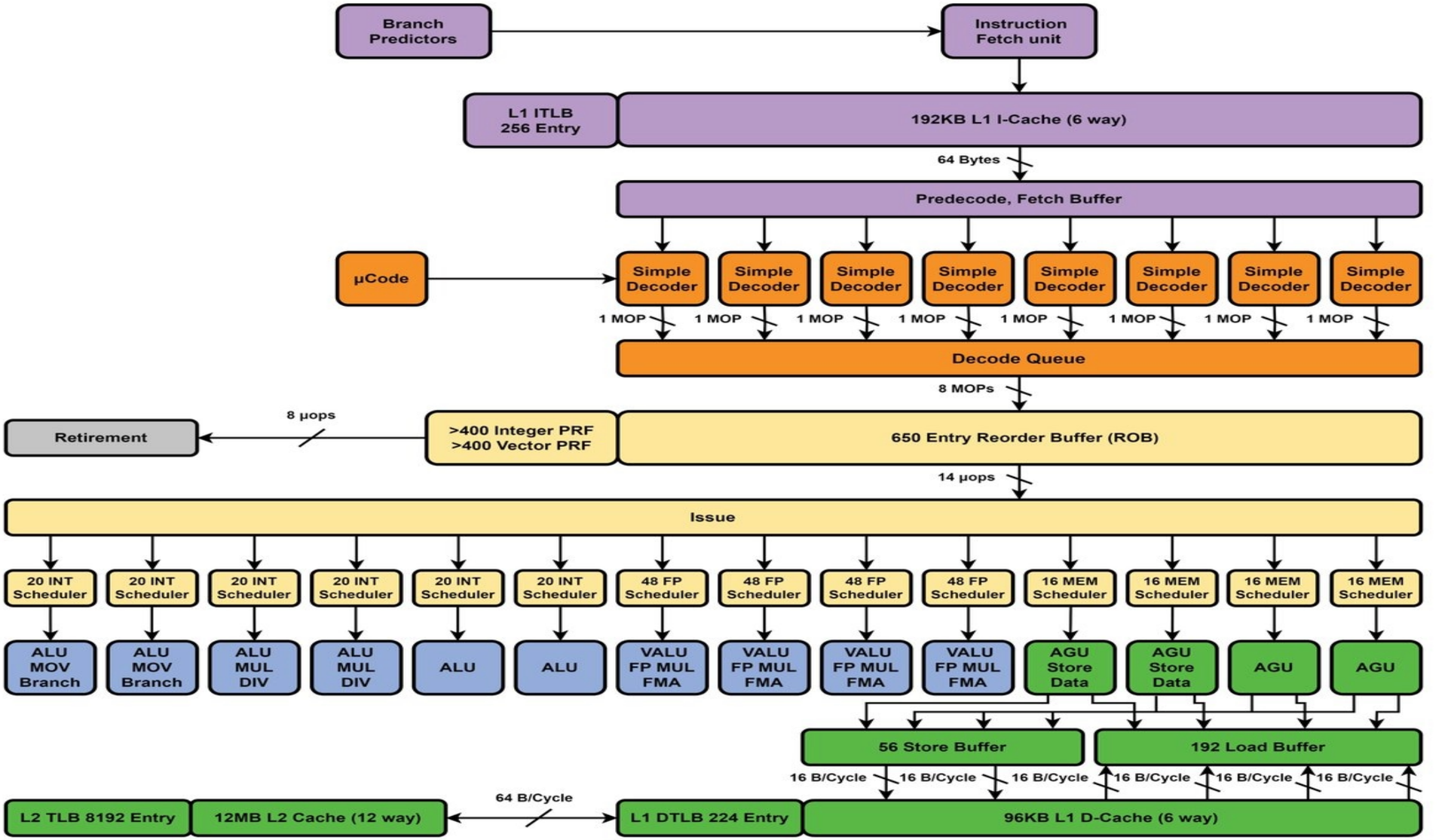


Qualcomm - 2024

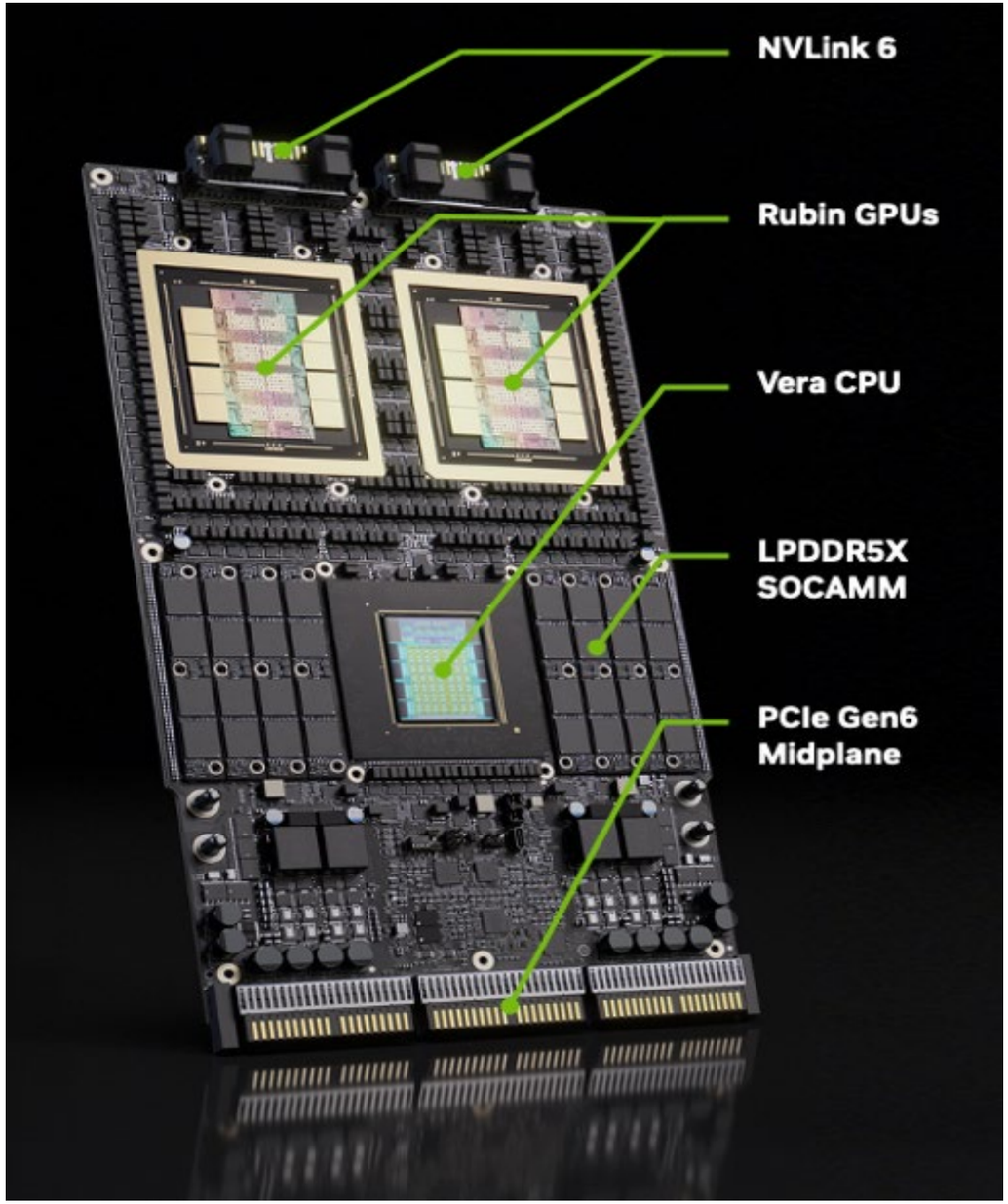
Oryon

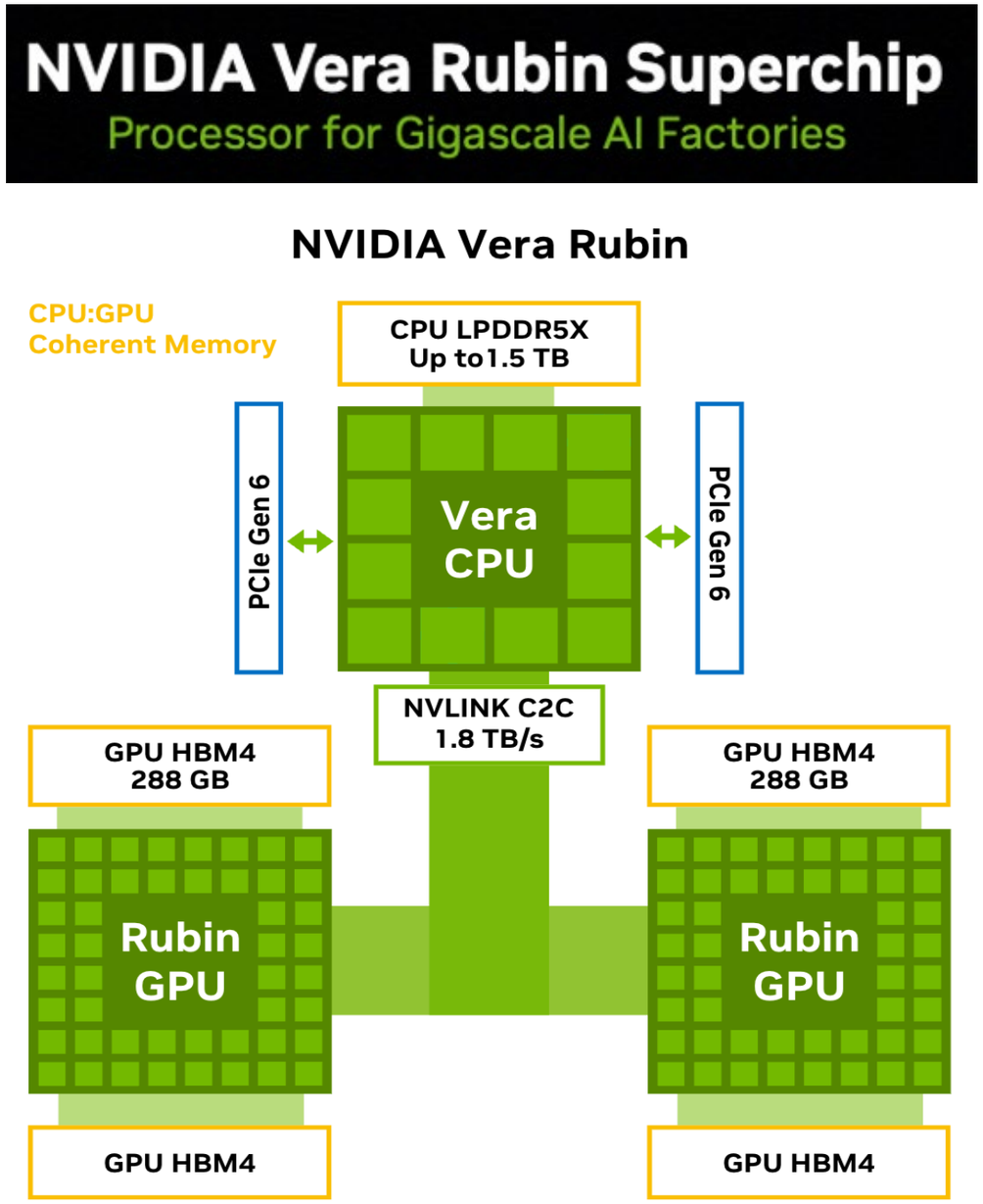
Microarchitecture Block Diagram

By Cardyak



https://developer.nvidia.com/blog/inside-the-nvidia-rubin-platform-six-new-chips-one-ai-supercomputer/

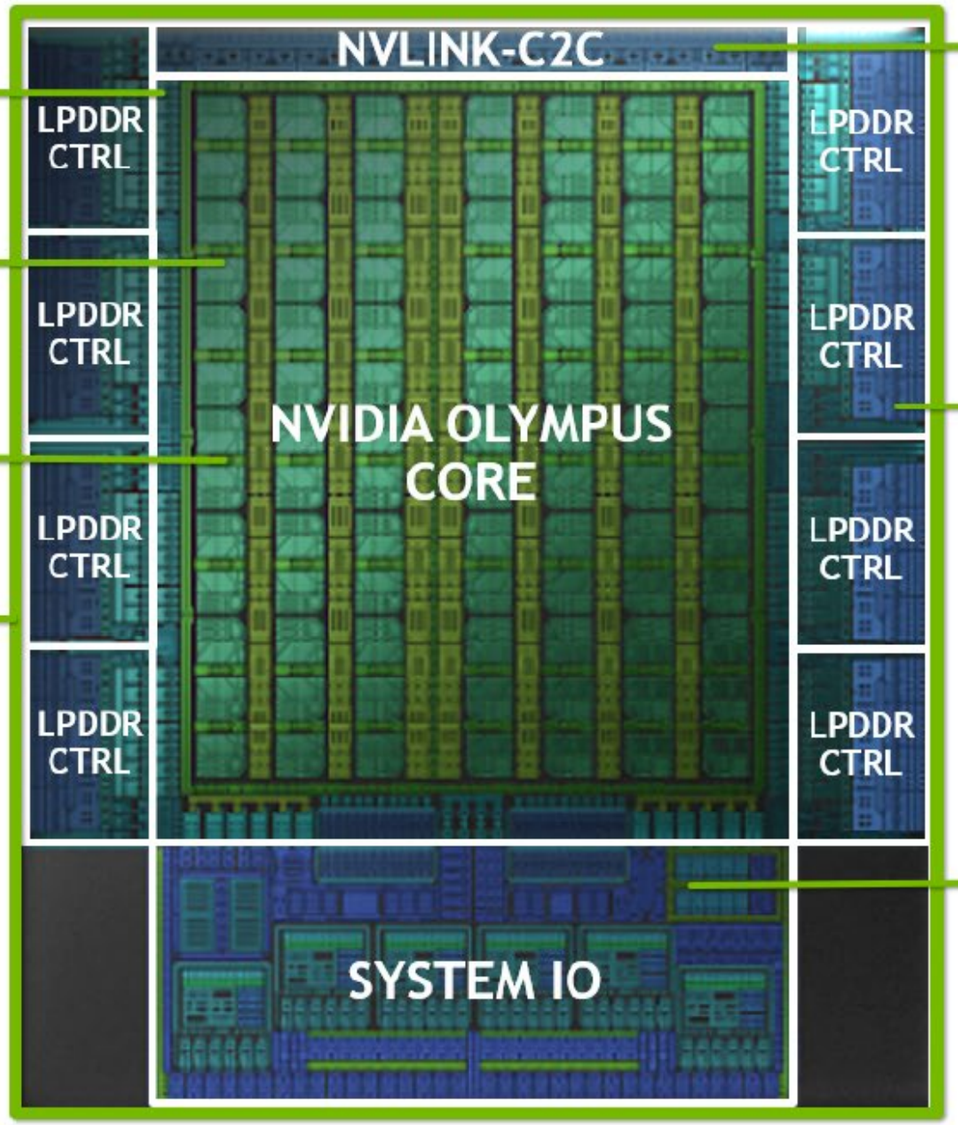




https://nvdam.widen.net/s/nmw5vblpqd/nvidia_vera_cpu_architecture_whitepaper?nvid=nv-tblg-543584

2026

NVIDIA Vera CPU



Monolithic Compute Die

88 NVIDIA Custom Olympus Cores with 176 threads 164MB L3 Cache

2nd Gen NVIDIA Scalable Coherency Fabric (SCF)

Confidential Computing TEE-I/O Capable

NVLink-C2C 1,800GB/s Coherent CPU-GPU Interface

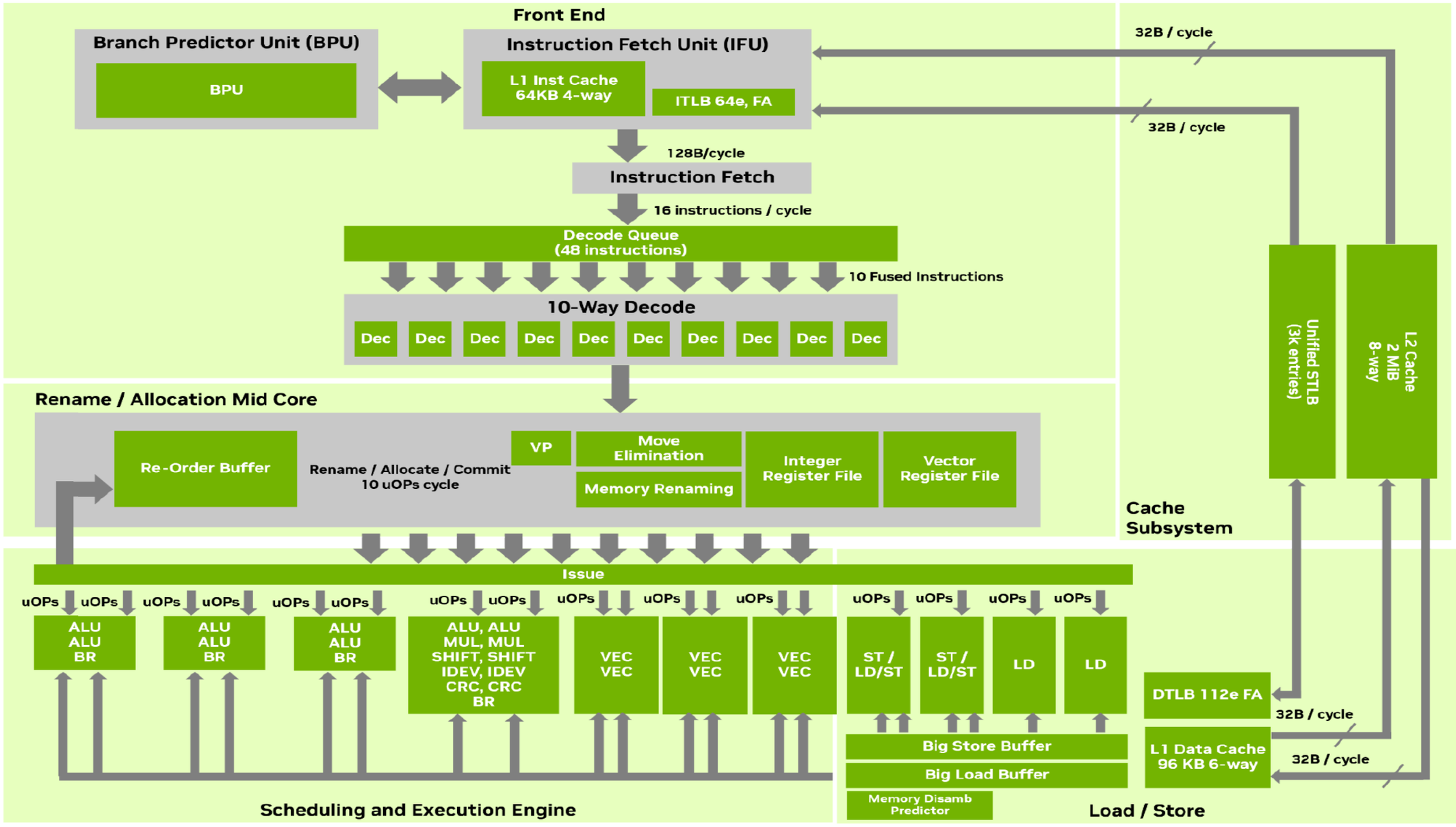
1.5TB SOCAMM LPDDR5X 1.2TB/s Memory BW

x16 PCIe Gen 6 CXL 3.1

https://nvdam.widen.net/s/nmw5vblpqd/nvidia_vera_cpu_architecture_whitepaper?nvid=nv-tblg-543584

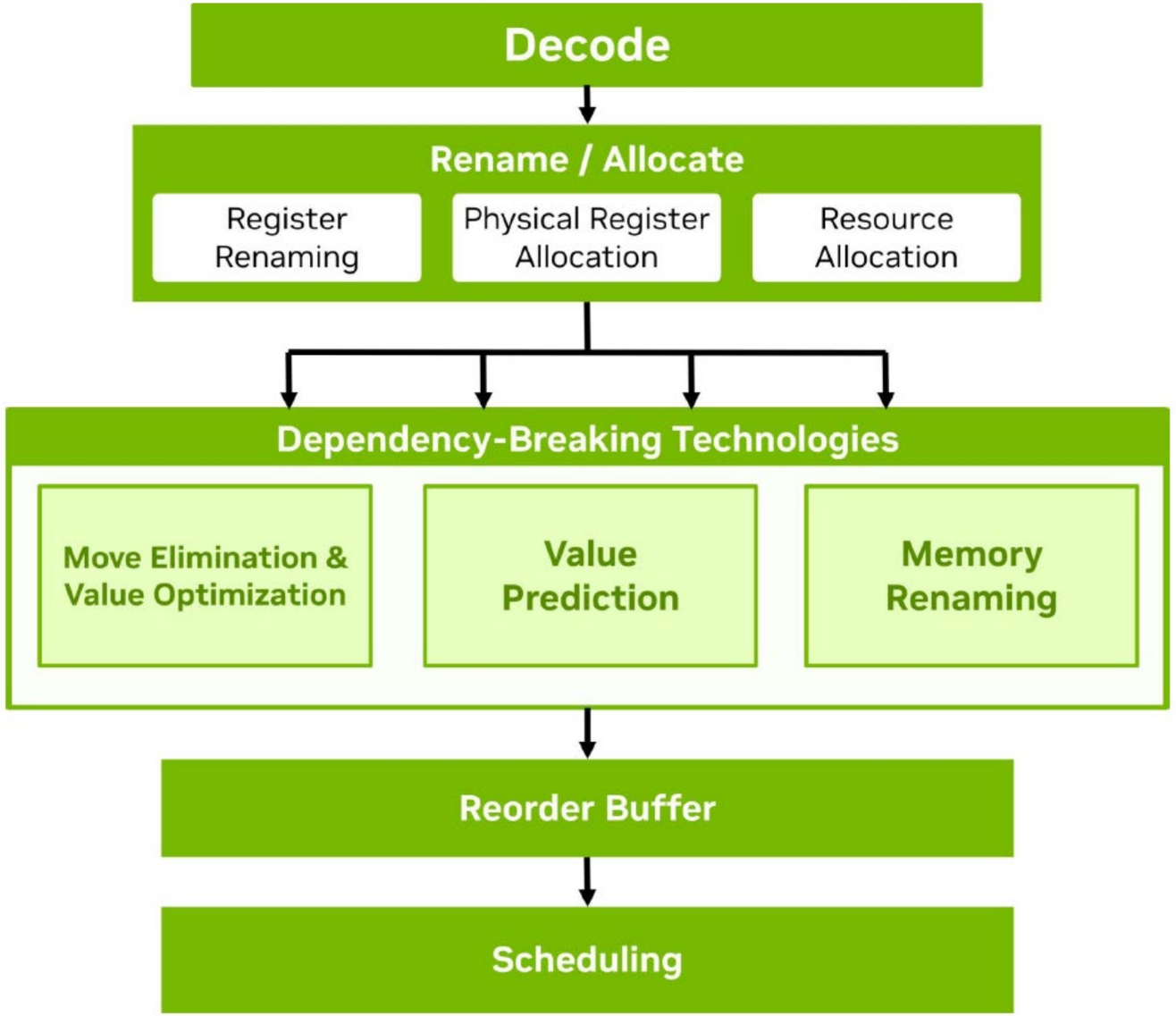
2026

NVIDIA Olympus Core Architecture



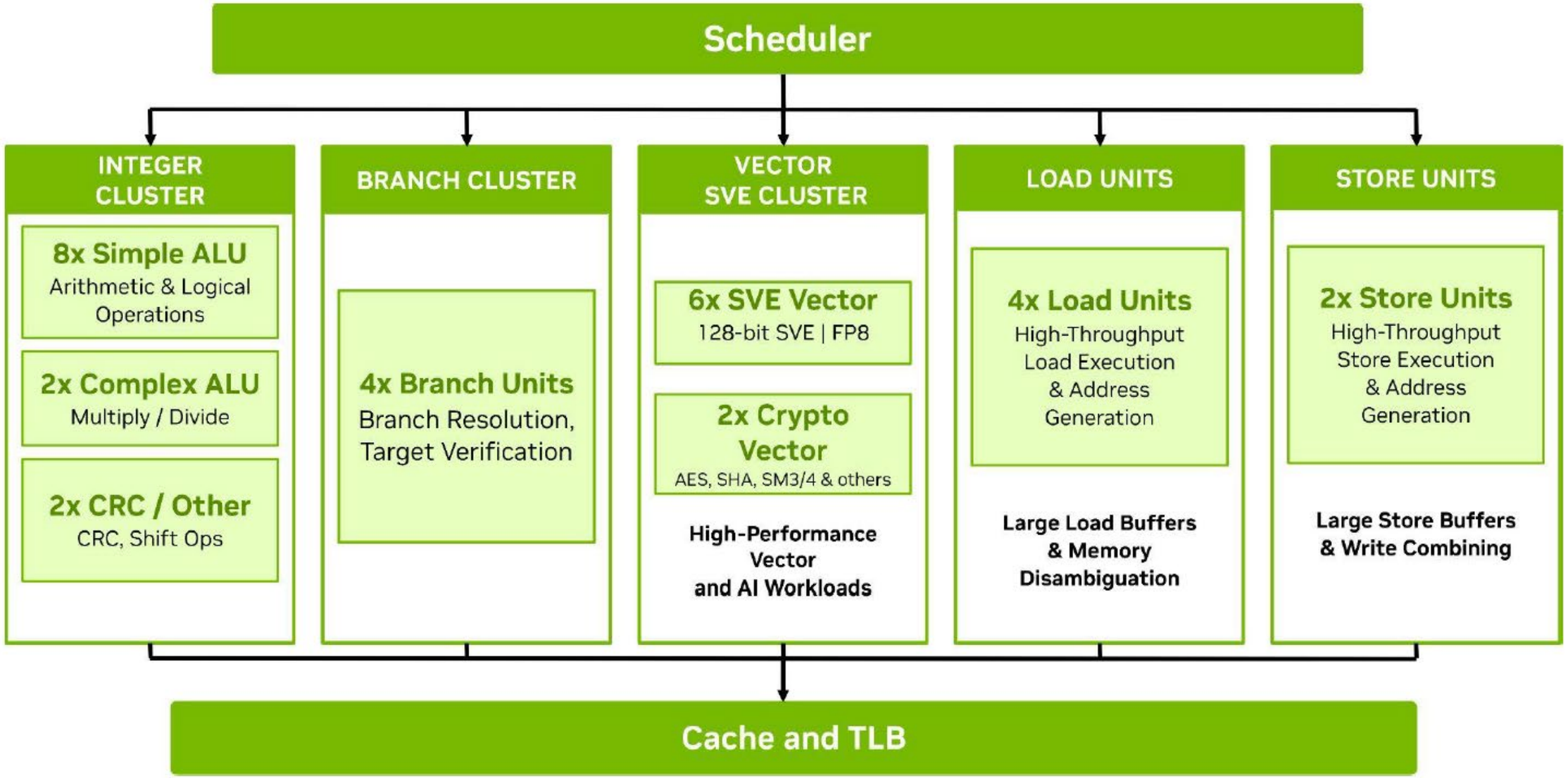
https://nvdam.widen.net/s/nmw5vblpqd/nvidia_vera_cpu_architecture_whitepaper?nvid=nv-tblg-543584

Mid Core – Critical Path Acceleration



https://nvdam.widen.net/s/nmw5vblpqd/nvidia_vera_cpu_architecture_whitepaper?nvid=nv-tblg-543584

Execution Engine – Complex Vector Optimization



18-740: Modern Computer Architecture & Design

Lecture 7:
"Modern Superscalar Out-of-Order Processors"

Prof. Olivia Hsu
September 15, 2026

Next Time ...

- Required Reading Assignment:
  - **Chapter 3 of Shen and Lipasti (SnL).**
- Recommended References:
  - Chapter 2 & Appendix B of Hennessey and Patterson 5th Edition

Electrical & Computer ENGINEERING